02 / SLIDE 001 Consigli di Programmazione e Elementi di Base per il Corso

Lezione: Strumenti, shell, Python e Git
Fonte: 02_strumenti_e_consigli.pdf, pagina 1 di 47

Spiegazione per studiare

La lezione raccoglie consigli pratici per impostare l'ambiente di sviluppo e gestire un progetto software. Gli argomenti principali saranno shell Unix, file system, Python, librerie, ambienti virtuali e versionamento con Git.

Definizioni, passaggi ed esempi presenti nella slide

Consigli di Programmazione e Elementi di Base per

il Corso

SLIDE ORIGINALE · diagrammi, tabelle e codice





Consigli di Programmazione e Elementi di Base per il Corso

02 / SLIDE 002 In questa lezione

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 2 di 47

Spiegazione per studiare

Per lavorare su un progetto Web servono sia strumenti di esecuzione sia pratiche di gestione del codice. La shell permette di controllare il sistema; Python e le librerie permettono di realizzare il backend; Git registra e condivide l'evoluzione del software.

Definizioni, passaggi ed esempi presenti nella slide

- Introduzione agli strumenti principali
- La shell UNIX
- Struttura della directory
- Esecuzione programmi base
- I file e come sono gestiti dal sistema operativo
- Breve introduzione a Python
- interpreti e librerie
- framework
- Gestione del codice
- versionamento
- piattaforme collaborative per il codice

SLIDE ORIGINALE · diagrammi, tabelle e codice

In questa lezione

- Introduzione agli strumenti principali
- La shell UNIX
- Struttura della directory
- Esecuzione programmi base
- I file e come sono gestiti dal sistema operativo
- Breve introduzione a Python
  - interpreti e librerie
  - framework
- Gestione del codice
  - versionamento
  - piattaforme collaborative per il codice

saiferlab.ai

2

02 / SLIDE 003 Requisiti di base per il corso: premessa

Lezione: Strumenti, shell, Python e Git
Fonte: 02_strumenti_e_consigli.pdf, pagina 3 di 47

Spiegazione per studiare

La docente sottolinea che le tecnologie e le librerie cambiano: conviene studiare i concetti invece di memorizzare meccanicamente ogni comando. La modularità rende più semplice sostituire una tecnologia senza riscrivere l'intera applicazione. Il collegamento con la programmazione a oggetti riguarda proprio riuso e separazione delle responsabilità.

Definizioni, passaggi ed esempi presenti nella slide

Ogni linguaggio di programmazione ha le sue caratteristiche e i suoi strumenti più usati

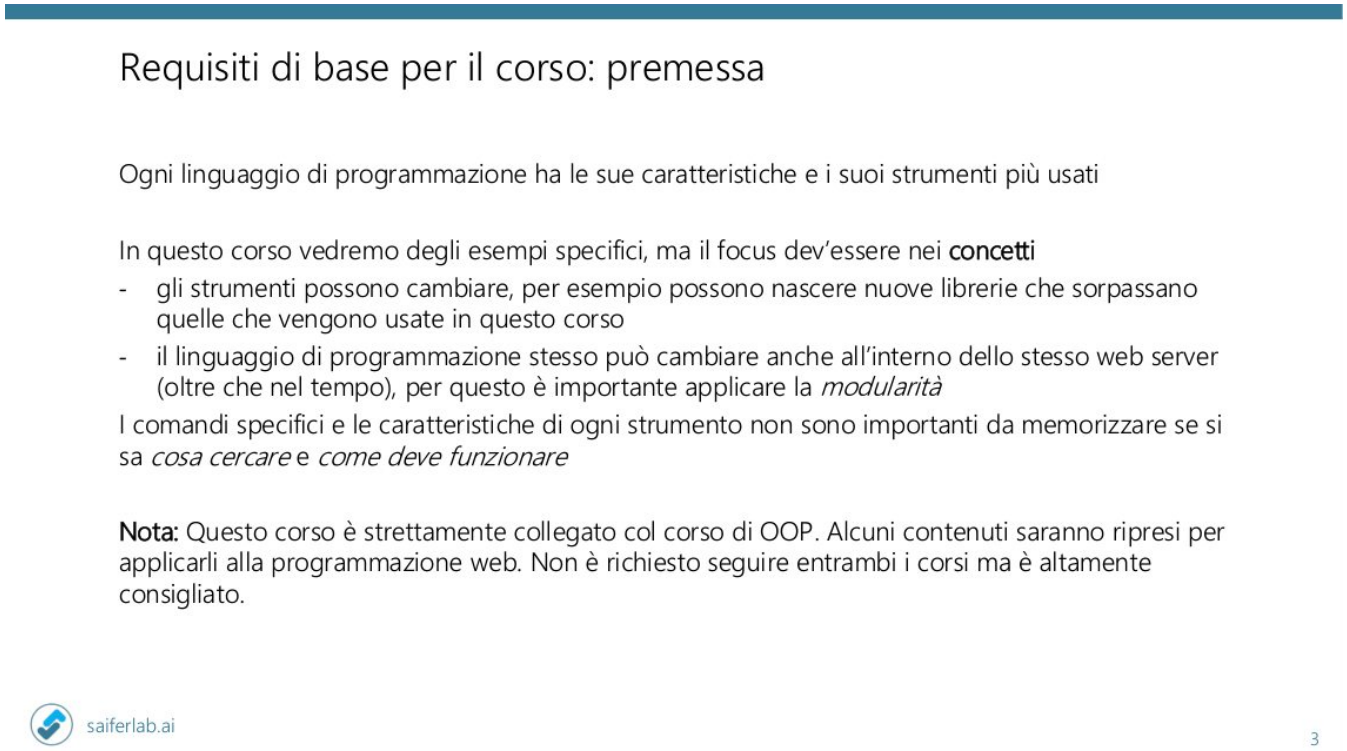
In questo corso vedremo degli esempi specifici, ma il focus dev'essere nei concetti

• gli strumenti possono cambiare, per esempio possono nascere nuove librerie che sorpassano quelle che vengono usate in questo corso

• il linguaggio di programmazione stesso può cambiare anche all'interno dello stesso web server (oltre che nel tempo), per questo è importante applicare la modularità I comandi specifici e le caratteristiche di ogni strumento non sono importanti da memorizzare se si sa cosa cercare e come deve funzionare

Nota: Questo corso è strettamente collegato col corso di OOP. Alcuni contenuti saranno ripresi per applicarli alla programmazione web. Non è richiesto seguire entrambi i corsi ma è altamente consigliato.

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 004 Requisiti di base per il corso

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 4 di 47

Spiegazione per studiare

Lo stack didattico combina un IDE per modificare e analizzare il codice, un terminale per lanciare comandi, Python per il backend, JavaScript per l'interfaccia e SQL per la persistenza. Ogni componente ha un ruolo diverso nella costruzione del server e nell'interazione con i dati.

Definizioni, passaggi ed esempi presenti nella slide

Durante il corso useremo degli strumenti per implementare un web server e le sue funzionalità

- Un ambiente di sviluppo integrato
- Il terminale
- Python principalmente per il backend
- JavaScript principalmente per il frontend
- SQL e un applicazione per database per le operazioni di lettura e scrittura su disco

Questa lezione ha lo scopo di dare le base su come impostare tutto (e le slide saranno la fonte primaria di informazioni nel caso in seguito serva qualcosa su questi tool)

SLIDE ORIGINALE · diagrammi, tabelle e codice

Requisiti di base per il corso

Durante il corso useremo degli strumenti per implementare un web server e le sue funzionalità
- Un ambiente di sviluppo integrato
- Il terminale
- Python principalmente per il backend
- JavaScript principalmente per il frontend
- SQL e un applicazione per database per le operazioni di lettura e scrittura su disco

Questa lezione ha lo scopo di dare le base su come impostare tutto (e le slide saranno la fonte primaria di informazioni nel caso in seguito serva qualcosa su questi tool)

02 / SLIDE 005 Integrated Development Environments (IDEs)

Lezione: Strumenti, shell, Python e Git
Fonte: 02_strumenti_e_consigli.pdf, pagina 5 di 47

Spiegazione per studiare

Un IDE integra editor, autocompletamento, debugger, strumenti di navigazione, versionamento delle dipendenze e gestione dell'interprete. Serve soprattutto nei progetti composti da più file e moduli, perché permette di trovare più rapidamente errori e collegamenti tra componenti. La slide cita Visual Studio Code e PyCharm come esempi.

Definizioni, passaggi ed esempi presenti nella slide

Per gestire progetti complessi (non script singoli), è consigliato utilizzare delle applicazioni che integrano più strumenti e consentono automazioni nello sviluppo

- editor di codice sorgente
- completamento automatico, strumenti di analisi del codice (es., per rilevare linee duplicate) e di navigazione rapida (es. salto alla definizione di funzioni o classi)
- debugger e logger
- controlli di versione delle librerie
- compilatori o gestori di interpreti
- ultimamente anche strumenti di AI Generativa per scrittura automatica del codice

I miei IDE suggeriti (ce ne sono tanti altri):

- Visual Studio Code https://code.visualstudio.com
- PyCharm e la suite IntelliJ https://www.jetbrains.com/pycharm/

SLIDE ORIGINALE · diagrammi, tabelle e codice

Integrated Development Environments (IDEs)

Per gestire progetti complessi (non script singoli), è consigliato utilizzare delle applicazioni che integrano più strumenti e consentono automazioni nello sviluppo

- editor di codice sorgente
- completamento automatico, strumenti di analisi del codice (es., per rilevare linee duplicate) e di navigazione rapida (es. salto alla definizione di funzioni o classi)
- debugger e logger
- controlli di versione delle librerie
- compilatori o gestori di interpreti
- ultimamente anche strumenti di AI Generativa per scrittura automatica del codice

I miei IDE suggeriti (ce ne sono tanti altri):

- **Visual Studio Code** https://code.visualstudio.com
- **PyCharm e la suite IntelliJ** https://www.jetbrains.com/pycharm/



saiferlab.ai

5

02 / SLIDE 006 La shell Unix e i principali comandi

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 6 di 47

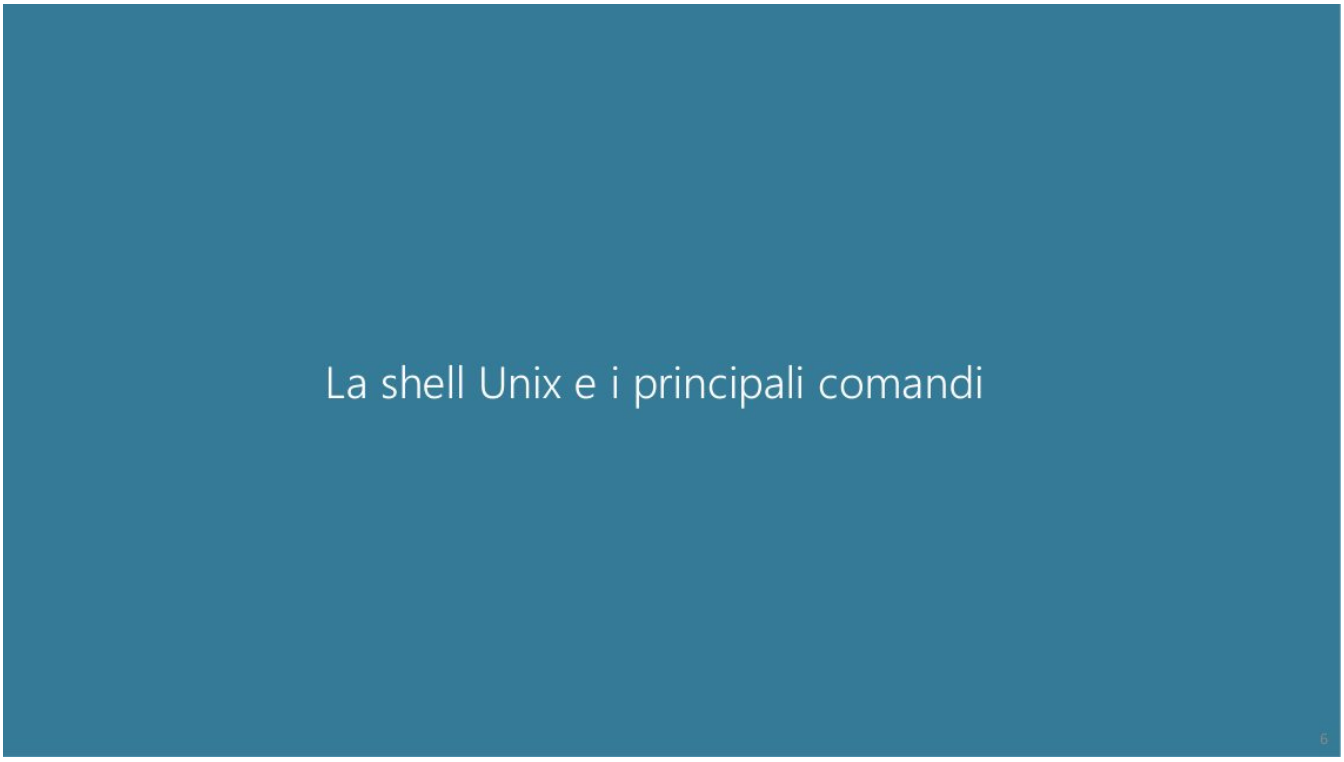
Spiegazione per studiare

Questa pagina introduce la sezione dedicata alla shell Unix e ai comandi principali. È uno spartiacque del materiale, non una slide che introduce nuove istruzioni operative.

Definizioni, passaggi ed esempi presenti nella slide

La shell Unix e i principali comandi

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 007 Il terminale virtuale (shell)

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 7 di 47

Spiegazione per studiare

Un terminale virtuale è l'applicazione attraverso cui si inseriscono comandi testuali e se ne leggono i risultati. Non coincide necessariamente con la shell: il terminale ne ospita l'interazione, mentre la shell interpreta i comandi. Le slide suggeriscono ambienti Unix oppure WSL su Windows.

Definizioni, passaggi ed esempi presenti nella slide

Un terminale virtuale (o emulatore di terminale), in informatica, è un programma o un servizio del sistema operativo che emula il comportamento di un terminale testuale

• dispositivo hardware, che viene usato per inserire dati in ingresso ad un computer e visualizzarli

in uscita

• in parole semplici il terminale virtuale è un’interfaccia a riga di comando per interagire con il

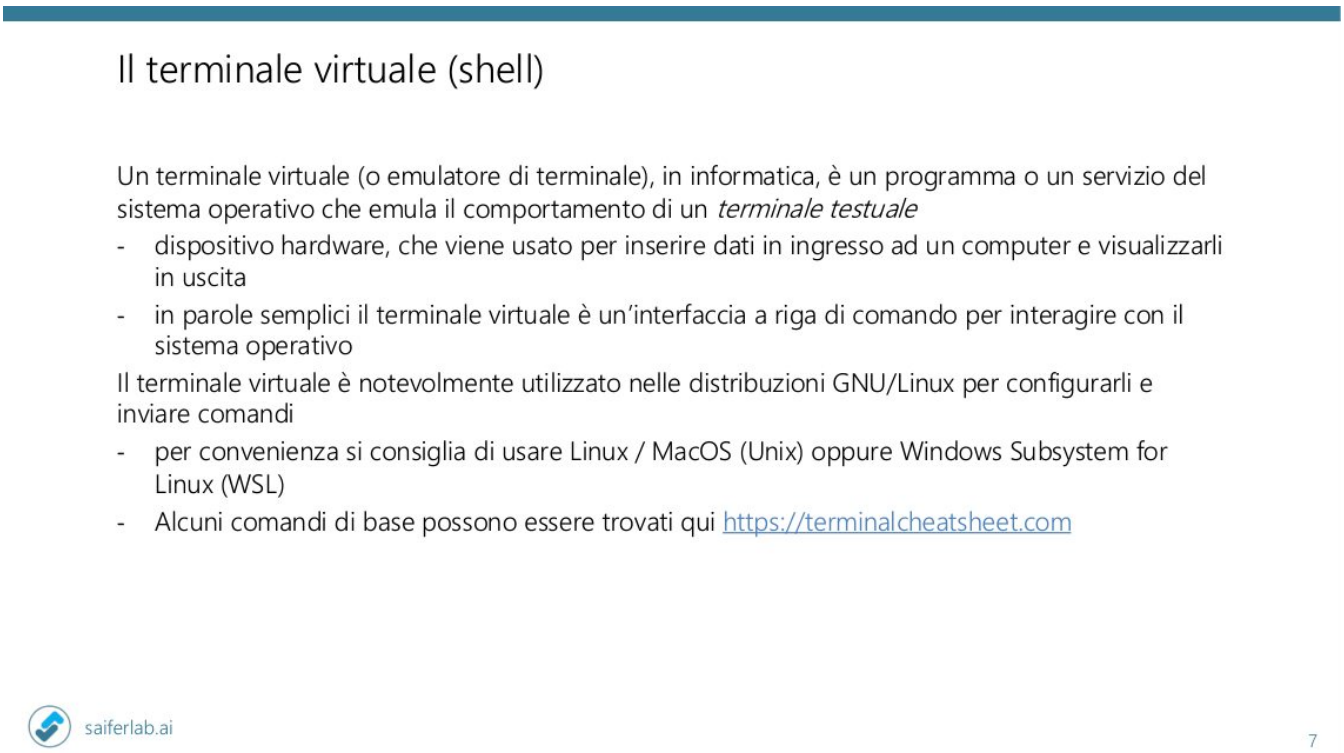
sistema operativo Il terminale virtuale è notevolmente utilizzato nelle distribuzioni GNU/Linux per configurarli e inviare comandi

• per convenienza si consiglia di usare Linux / MacOS (Unix) oppure Windows Subsystem for

Linux (WSL)

• Alcuni comandi di base possono essere trovati qui https://terminalcheatsheet.com

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 008 La shell UNIX

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 8 di 47

Spiegazione per studiare

La shell è un'interfaccia testuale alternativa alla GUI. Risulta meno immediata per chi inizia, ma favorisce controllo, ripetibilità e automazione: si possono concatenare comandi per compiere elaborazioni che con il mouse richiederebbero molti passaggi.

Definizioni, passaggi ed esempi presenti nella slide

La shell è un programma, alternativo alle classiche interfacce grafiche (Graphical User Interface, GUI), in cui gli utenti possono inserire dei compandi per il computer

Usare la shell richiede un po' di sforzo in più per imparare i comandi ... ma da molto più controllo sulle operazioni e consente maggiori automazioni

• l'utilizzo è guidato da una serie di istruzioni testuali

Le GUI sono più facili da usare per gli utenti non esperti, in quanto nascondono i comandi

• l'utilizzo è guidato da comandi del mouse e shortcut da tastiera

La grammatica dei comandi shell permette di combinare strumenti esistenti in potenti pipeline, e facilita la gestione automatica di più larghi volumi di dati

SLIDE ORIGINALE · diagrammi, tabelle e codice

La shell UNIX

La *shell* è un programma, alternativo alle classiche interfacce grafiche (Graphical User Interface, GUI), in cui gli utenti possono inserire dei compandi per il computer

Usare la shell richiede un po' di sforzo in più per imparare i comandi
... ma da molto più controllo sulle operazioni e consente maggiori automazioni
- l'utilizzo è guidato da una serie di istruzioni testuali

Le GUI sono più facili da usare per gli utenti non esperti, in quanto nascondono i comandi
- l'utilizzo è guidato da comandi del mouse e shortcut da tastiera

La grammatica dei comandi shell permette di combinare strumenti esistenti in potenti *pipeline*, e facilita la gestione automatica di più larghi volumi di dati

saiferlab.ai

8

02 / SLIDE 009 Perché le GUI non bastano

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 9 di 47

Spiegazione per studiare

L'esempio dei cento file numerati mostra perché la shell è utile per le operazioni ripetitive. Eseguire la procedura manualmente è lento ed espone a errori; una sequenza di comandi consente di riprodurla in modo consistente. Il principio importante è automatizzare compiti deterministici.

Definizioni, passaggi ed esempi presenti nella slide

Nonostante le GUI siano molto facili da imparare a usare, limitandosi al loro utilizzo si rischia di dover fare molto più lavoro

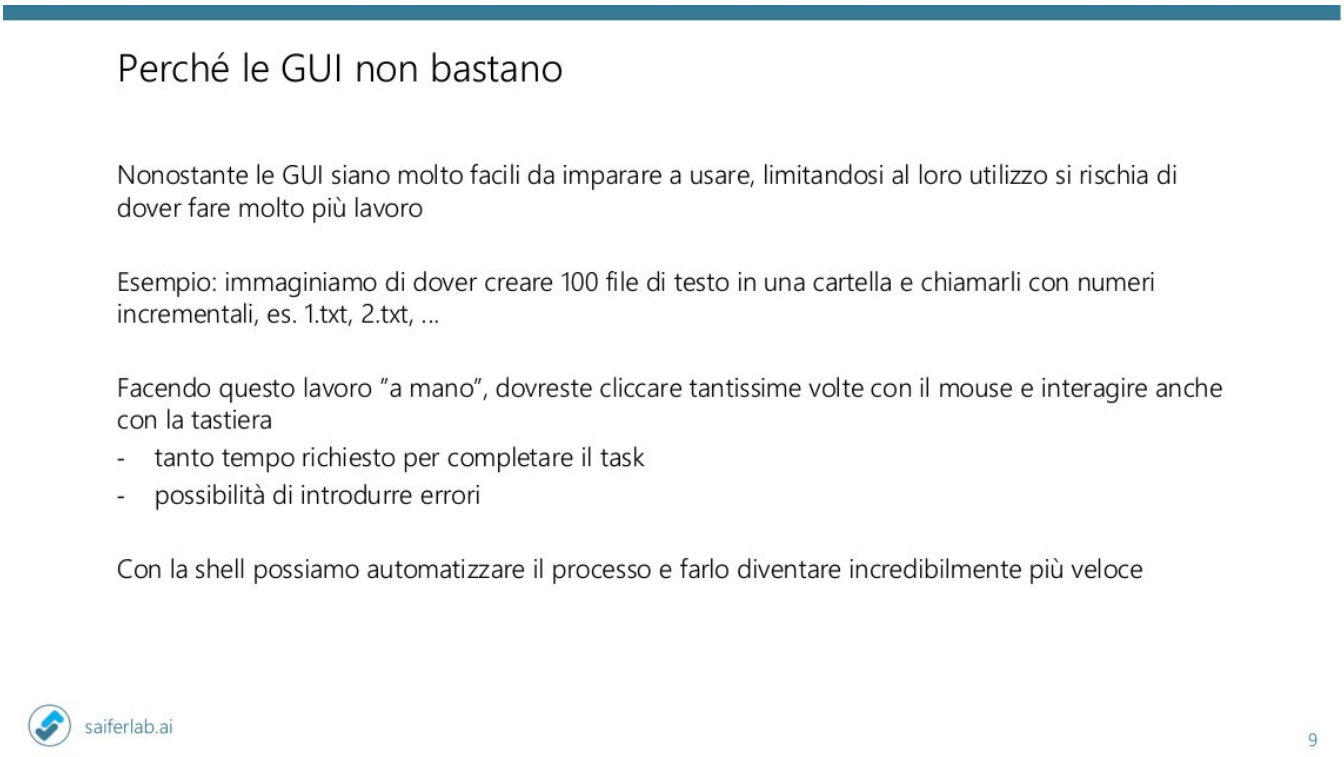
Esempio: immaginiamo di dover creare 100 file di testo in una cartella e chiamarli con numeri incrementali, es. 1.txt, 2.txt, ...

Facendo questo lavoro ”a mano”, dovreste cliccare tantissime volte con il mouse e interagire anche con la tastiera

- tanto tempo richiesto per completare il task
- possibilità di introdurre errori

Con la shell possiamo automatizzare il processo e farlo diventare incredibilmente più veloce

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 010 Fare sempre attenzione ai comandi che si danno alla shell...

Lezione: Strumenti, shell, Python e Git
Fonte: 02_strumenti_e_consigli.pdf, pagina 10 di 47

Spiegazione per studiare

I comandi della shell possono modificare o cancellare dati in modo irreversibile, soprattutto se eseguiti con privilegi elevati. La slide usa `rm -rf /` come esempio da non eseguire: prima di lanciare un comando occorre comprendere gli argomenti e la directory interessata.

Definizioni, passaggi ed esempi presenti nella slide

Fare sempre attenzione ai comandi che si danno alla shell...

Evitate il famoso

rm –rf /

Non date assolutamente questo comando!!! Tra poco saremo in grado di capirlo e sapremo perché si deve evitare

Per ora limitatevi ai comandi usati in questo tutorial finché non capite bene cosa significano

SLIDE ORIGINALE · diagrammi, tabelle e codice

Fare sempre attenzione ai comandi che si danno alla shell...

Fare sempre attenzione ai comandi che si danno alla shell...

Evitate il famoso

```
rm -rf /
```

Non date assolutamente questo comando!!!
Tra poco saremo in grado di capirlo e sapremo perché si deve evitare

Per ora limitatevi ai comandi usati in questo tutorial finché non capite bene cosa significano



saiferlab.ai

10

02 / SLIDE 011 Altri usi utili della shell

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 11 di 47

Spiegazione per studiare

La shell consente anche di controllare computer remoti, senza accedervi fisicamente. Per un'applicazione Web questo è importante perché il codice deve spesso essere copiato, configurato e avviato sul server che ospita il servizio. Il diagramma raffigura il collegamento verso macchine remote tramite shell e SSH.

Definizioni, passaggi ed esempi presenti nella slide

La shell è anche molto utile per interagire da remoto con altri computer

Prendere familiarità con la shell ci permette di usare una serie di strumenti e risorse specializzate, inclusi computer ad alte prestazioni, senza bisogno di essere connessi fisicamente

Nel contesto della programmazione web, è molto utile saper usare la shell per copiare il codice del nostro web server e farlo girare da remoto

SLIDE ORIGINALE · diagrammi, tabelle e codice

Altri usi utili della shell

La shell è anche molto utile per interagire da remoto con altri computer

Prendere familiarità con la shell ci permette di usare una serie di strumenti e risorse specializzate, inclusi computer ad alte prestazioni, senza bisogno di essere connessi fisicamente

Nel contesto della programmazione web, è molto utile saper usare la shell per copiare il codice del nostro web server e farlo girare *da remoto*



saiferlab.ai

11

02 / SLIDE 012 Comandi di base con la shell

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 12 di 47

Spiegazione per studiare

Bash è presentata come una shell Unix e come linguaggio di comandi. Quando si invia una riga, la shell la interpreta e avvia le operazioni richieste. Questo consente anche la scrittura di script, cioè sequenze di istruzioni riutilizzabili.

Definizioni, passaggi ed esempi presenti nella slide

Vediamo alcuni comandi di base con la shell

Bash è una shell Unix e anche un linguaggio per istruzioni, impostato come default per Linux e MacOS

Si tratta di un linguaggio interpretato, che si contrappone ai linguaggi compilati

Maggiori dettagli sui linguaggi interpretati saranno dati in seguito quando parleremo di Python (anch'esso un linguaggio interpretato)

SLIDE ORIGINALE · diagrammi, tabelle e codice

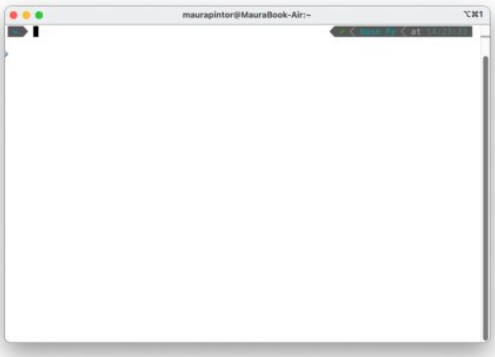
Comandi di base con la shell

Vediamo alcuni comandi di base con la shell

Bash è una shell Unix e anche un linguaggio per istruzioni, impostato come default per Linux e MacOS

Si tratta di un linguaggio *interpretato*, che si contrappone ai linguaggi *compilati*

Maggiori dettagli sui linguaggi interpretati saranno dati in seguito quando parleremo di Python (anch'esso un linguaggio interpretato)



saiferlab.ai

12

02 / SLIDE 013 Primi comandi alla shell

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 13 di 47

Spiegazione per studiare

Per imparare a usare la shell conviene consultare i manuali dei comandi, invece di procedere per tentativi. La documentazione specifica sintassi, opzioni e argomenti dei programmi da riga di comando. L'obiettivo della slide è acquisire autonomia nella consultazione degli strumenti.

Definizioni, passaggi ed esempi presenti nella slide

Per iniziare, possiamo chiedere alla shell il manuale di alcuni comandi

I comandi sono programmi pre- esistenti che possiamo richiamare usano il loro nome

Per esempio, il comando ls serve a fare la lista dei file in una directory

Possiamo accedere al manuale (anche esso un programma) scrivendo man ls

SLIDE ORIGINALE · diagrammi, tabelle e codice

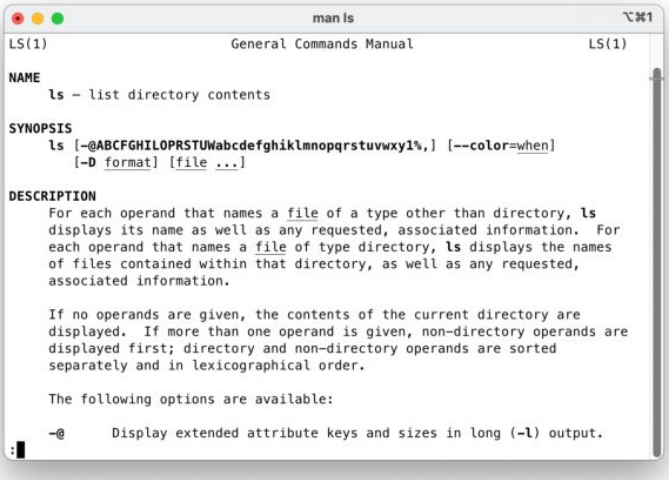
Primi comandi alla shell

Per iniziare, possiamo chiedere alla shell il manuale di alcuni comandi

I comandi sono programmi pre-esistenti che possiamo richiamare usano il loro nome

Per esempio, il comando `ls` serve a fare la lista dei file in una directory

Possiamo accedere al manuale (anche esso un programma) scrivendo `man ls`



saiferlab.ai

13

02 / SLIDE 014 Eseguire programmi nella shell

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 14 di 47

Spiegazione per studiare

Un programma può essere avviato digitando il percorso del relativo eseguibile. La shell interpreta la riga e avvia l'esecuzione quando l'utente preme Invio. Il path determina precisamente quale programma verrà avviato, evitando ambiguità sui nomi.

Definizioni, passaggi ed esempi presenti nella slide

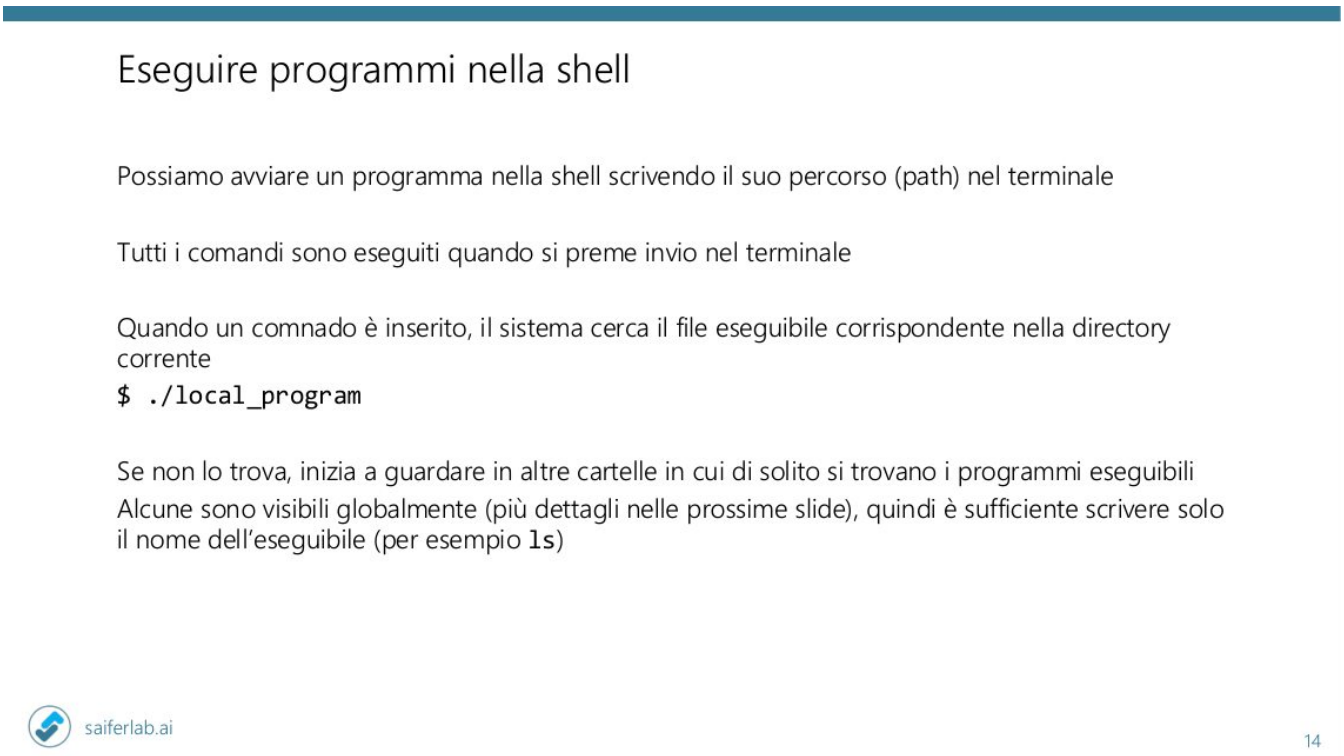
Possiamo avviare un programma nella shell scrivendo il suo percorso (path) nel terminale

Tutti i comandi sono eseguiti quando si preme invio nel terminale

Quando un comnado è inserito, il sistema cerca il file eseguibile corrispondente nella directory corrente $ ./local_program

Se non lo trova, inizia a guardare in altre cartelle in cui di solito si trovano i programmi eseguibili Alcune sono visibili globalmente (più dettagli nelle prossime slide), quindi è sufficiente scrivere solo il nome dell’eseguibile (per esempio ls)

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 015 Dove si trovano i programmi della shell?

Lezione: Strumenti, shell, Python e Git
Fonte: 02_strumenti_e_consigli.pdf, pagina 15 di 47

Spiegazione per studiare

Quando si digita soltanto il nome di un comando, la shell lo cerca nelle directory contenute nella variabile PATH. Per questo un eseguibile può essere avviato anche senza specificare il percorso completo. Capire la risoluzione del percorso è utile quando un comando risulta installato ma non viene trovato.

Definizioni, passaggi ed esempi presenti nella slide

I programmi della shell (inclusi man, ls), sono chiamati tramite il nome del loro eseguibile

La shell definisce una variabile chiamata $PATH, che include le directory dove si trovano tutti i programmi eseguibili dalla shell

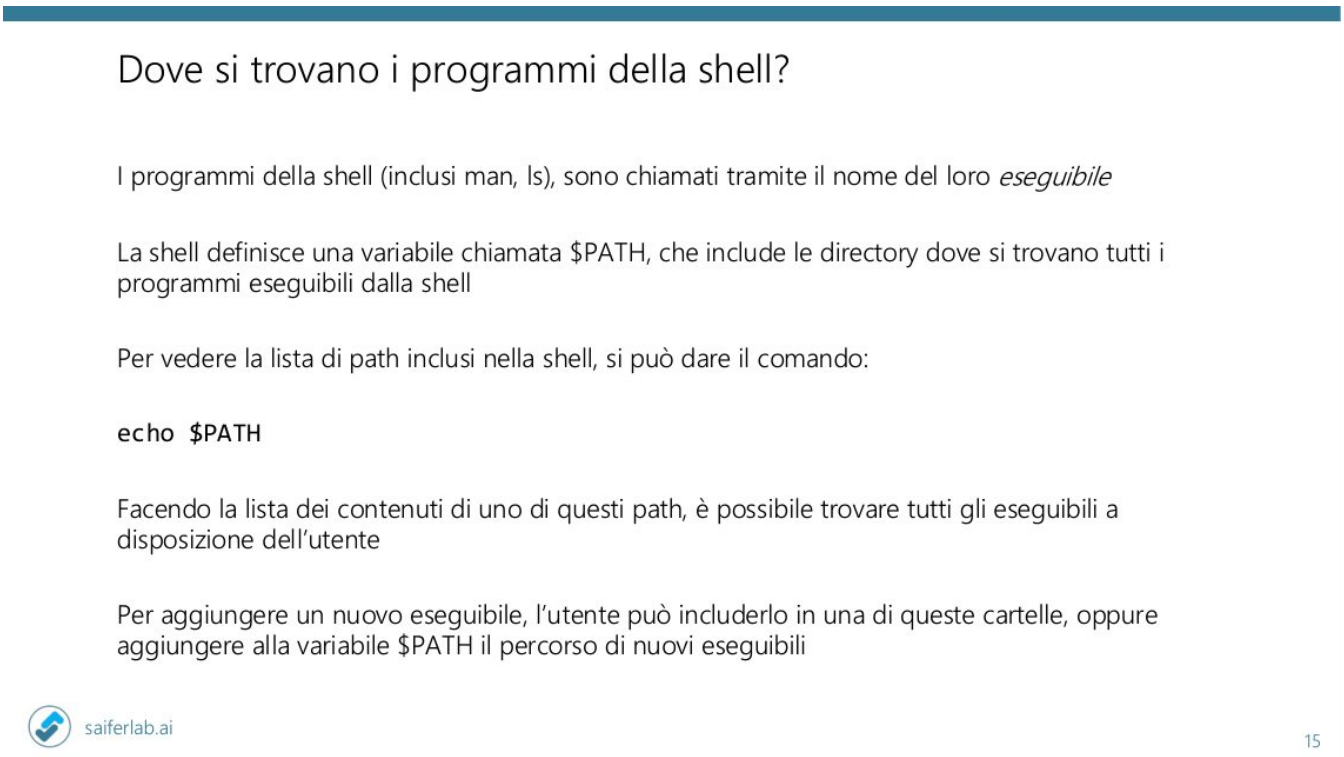
Per vedere la lista di path inclusi nella shell, si può dare il comando:

echo $PATH

Facendo la lista dei contenuti di uno di questi path, è possibile trovare tutti gli eseguibili a disposizione dell'utente

Per aggiungere un nuovo eseguibile, l'utente può includerlo in una di queste cartelle, oppure aggiungere alla variabile $PATH il percorso di nuovi eseguibili

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 016 I File nel nostro sistema

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 16 di 47

Spiegazione per studiare

Un file è un insieme di dati memorizzato in modo persistente; il sistema operativo lo descrive con metadati e permessi. I file possono essere ordinati in directory e gestiti tramite comandi della shell. Questa struttura rende possibile distinguere codice, configurazioni e risorse di un'applicazione.

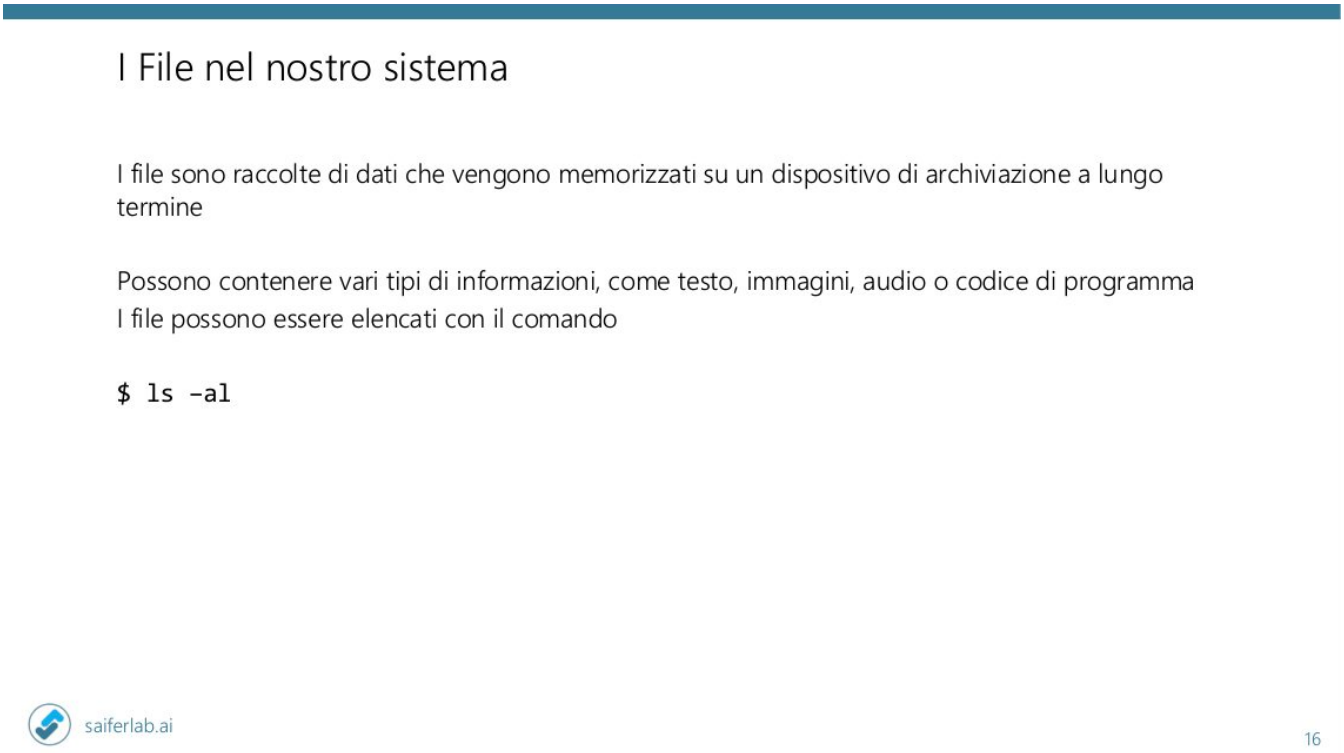
Definizioni, passaggi ed esempi presenti nella slide

I file sono raccolte di dati che vengono memorizzati su un dispositivo di archiviazione a lungo termine

Possono contenere vari tipi di informazioni, come testo, immagini, audio o codice di programma I file possono essere elencati con il comando

$ ls –al

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 017 Esempio di output del comando ls — panoramica

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 17 di 47

Spiegazione per studiare

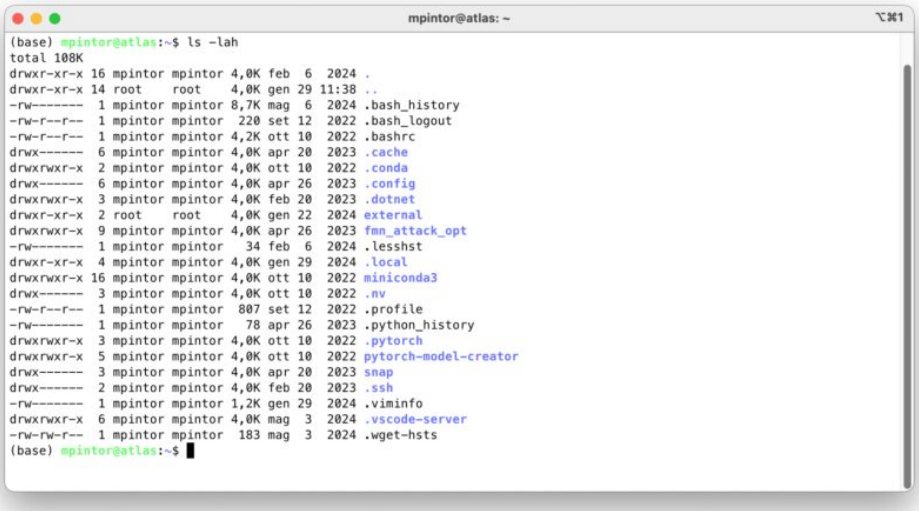
La slide mostra un esempio visivo dell'output di `ls`, il comando con cui si elenca il contenuto di una directory. La lettura di questo output permette di capire quali file sono presenti e quali informazioni aggiuntive sono disponibili quando si usano le opzioni appropriate.

Definizioni, passaggi ed esempi presenti nella slide

Questa slide non contiene testo estraibile; la figura originale è disponibile nel riferimento visivo.

SLIDE ORIGINALE · diagrammi, tabelle e codice

Esempio di output del comando `ls`



saiferlab.ai

17

02 / SLIDE 018 Esempio di output del comando ls — permessi

Lezione: Strumenti, shell, Python e Git
Fonte: 02_strumenti_e_consigli.pdf, pagina 18 di 47

Spiegazione per studiare

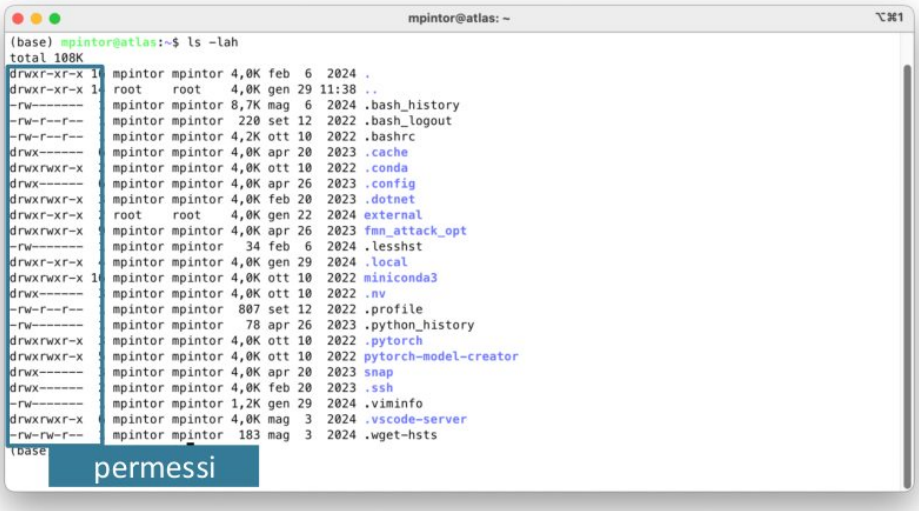
Nell'output dettagliato di `ls`, il campo dei permessi indica quali operazioni sono consentite sul file o sulla directory. Le autorizzazioni di lettura, scrittura ed esecuzione sono distribuite tra proprietario, gruppo e altri utenti. L'immagine mette in evidenza questa parte della riga.

Definizioni, passaggi ed esempi presenti nella slide

permessi

SLIDE ORIGINALE · diagrammi, tabelle e codice

Esempio di output del comando `ls`



saiferlab.ai

18

02 / SLIDE 019 Esempio di output del comando ls — proprietario e gruppo

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 19 di 47

Spiegazione per studiare

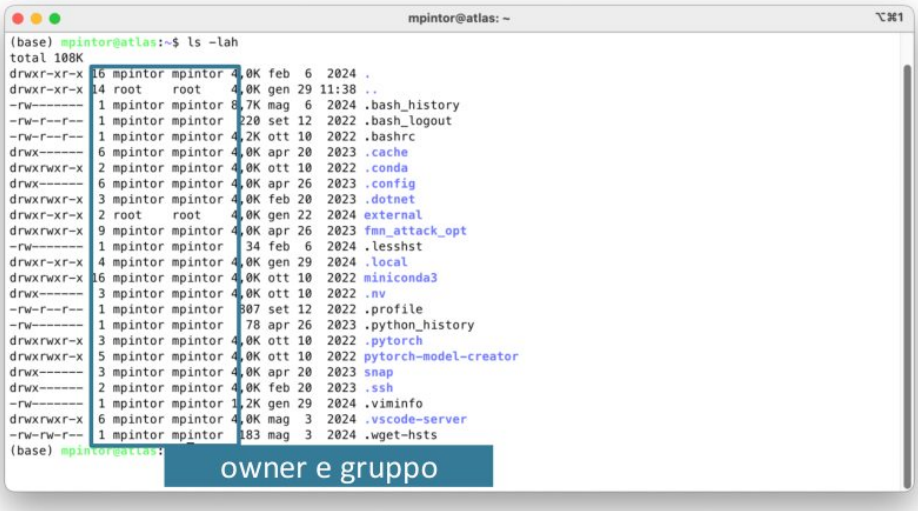
L'output di `ls` presenta anche proprietario e gruppo associati a ciascun elemento. Queste informazioni servono a capire quali identità vengono prese in considerazione dal sistema nel decidere se concedere accesso a file e directory.

Definizioni, passaggi ed esempi presenti nella slide

owner e gruppo

SLIDE ORIGINALE · diagrammi, tabelle e codice

Esempio di output del comando `ls`



saiferlab.ai

19

02 / SLIDE 020 Esempio di output del comando ls — dimensione

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 20 di 47

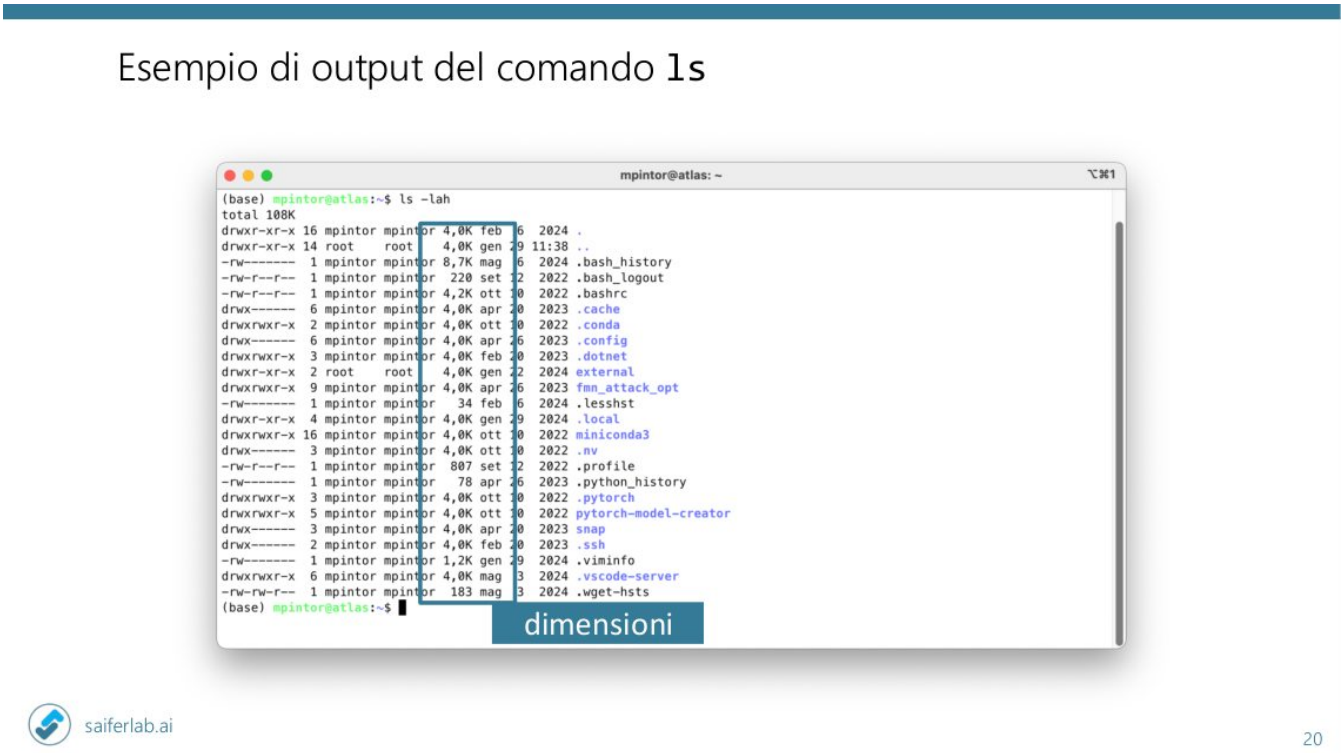
Spiegazione per studiare

Il valore relativo alla dimensione descrive quanto spazio occupa il file, espresso nel formato fornito dal comando. Questo campo è utile per distinguere rapidamente file piccoli, file grandi e potenziali anomalie in una cartella.

Definizioni, passaggi ed esempi presenti nella slide

dimensioni

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 021 Esempio di output del comando ls — ultima modifica

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 21 di 47

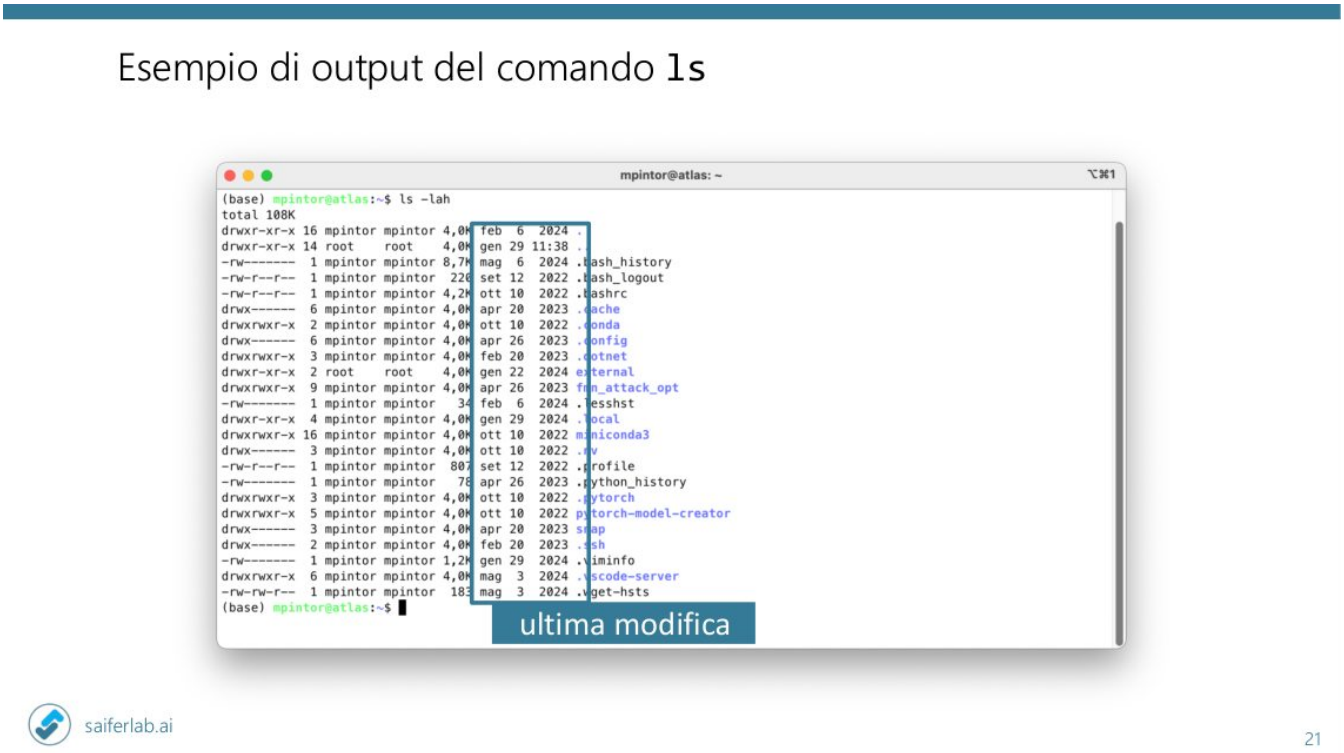
Spiegazione per studiare

La data o l'ora dell'ultima modifica permettono di riconoscere quali file sono stati aggiornati recentemente. Nei flussi di sviluppo aiutano a controllare quando sono cambiati codice, configurazioni o risultati di un'esecuzione.

Definizioni, passaggi ed esempi presenti nella slide

ultima modifica

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 022 Esempio di output del comando ls — nome file

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 22 di 47

Spiegazione per studiare

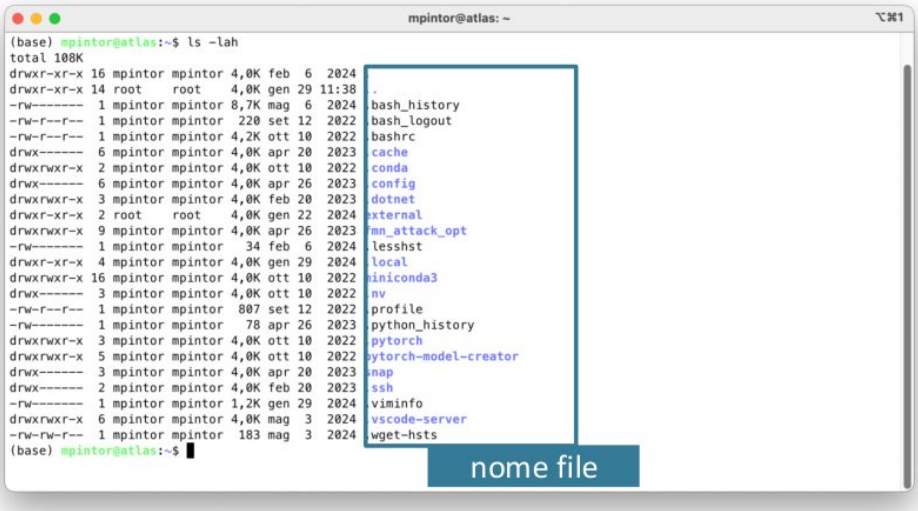
L'ultima parte della riga restituita da `ls` contiene il nome del file o della directory. Associando nome e metadati visualizzati nei campi precedenti si può identificare correttamente ciascun oggetto del file system.

Definizioni, passaggi ed esempi presenti nella slide

nome file

SLIDE ORIGINALE · diagrammi, tabelle e codice

Esempio di output del comando `ls`



saiferlab.ai

22

02 / SLIDE 023 I permessi dei file

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 23 di 47

Spiegazione per studiare

I permessi Unix sono ripartiti in tre gruppi: utente proprietario, gruppo e altri. Ciascun gruppo può avere combinazioni di lettura (`r`), scrittura (`w`) ed esecuzione (`x`). Questa distinzione è fondamentale quando il server non riesce ad accedere a un file o quando bisogna limitarne l'esposizione.

Definizioni, passaggi ed esempi presenti nella slide

Il primo gruppo di autorizzazioni si applica al proprietario del file Il secondo gruppo di autorizzazioni si applica al gruppo di utenti che possiede il file Il terzo gruppo di autorizzazioni viene generalmente definito “altri”

Ogni carattere dell'espressione indica se un permesso specifico è concesso o meno

- permesso di lettura (r)
- permesso di scrittura (w)
- permesso di esecuzione (x)

SLIDE ORIGINALE · diagrammi, tabelle e codice

I permessi dei file

Il primo gruppo di autorizzazioni si applica al **proprietario** del file
Il secondo gruppo di autorizzazioni si applica al **gruppo** di utenti che possiede il file
Il terzo gruppo di autorizzazioni viene generalmente definito “**altri**”

Ogni carattere dell'espressione indica se un permesso specifico è concesso o meno
- permesso di lettura (r)
- permesso di scrittura (w)
- permesso di esecuzione (x)

```
drwx------  6 mpintor mpintor 4,0K apr 26  2023 .config
drwxrwxr-x  3 mpintor mpintor 4,0K feb 20  2023 .dotnet
drwxr-xr-x  2 root    root    4,0K gen 22  2024 external
drwxrwxr-x  9 mpintor mpintor 4,0K apr 26  2023 fmn_attack_opt
-rw-------  1 mpintor mpintor   34 feb  6  2024 .lesshst
drwxr-xr-x  4 mpintor mpintor 4,0K gen 29  2024 .local
drwxrwxr-x 16 mpintor mpintor 4,0K ott 10  2022 miniconda3
drwx------  3 mpintor mpintor 4,0K ott 10  2022 .nv
-rw-r--r--  1 mpintor mpintor  807 set 12  2022 .profile
-rw-------  1 mpintor mpintor   78 apr 26  2023 .python_history
drwxrwxr-x  3 mpintor mpintor 4,0K ott 10  2022 .pytorch
drwxrwxr-x  5 mpintor mpintor 4,0K ott 10  2022 pytorch-model-creator
drwx------  3 mpintor mpintor 4,0K apr 20  2023 snap
```

saiferlab.ai

23

02 / SLIDE 024 I file

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 24 di 47

Spiegazione per studiare

Il file system Unix è organizzato gerarchicamente a partire dalla directory radice /. Le directory possono contenere file e altre directory, formando un albero. Percorsi assoluti e relativi identificano la posizione degli elementi e sono usati in molti comandi di gestione del progetto.

Definizioni, passaggi ed esempi presenti nella slide

I file contengono altri file, che si diramano dalla radice “/” formando una gerarchia ad albero

- Si trovano in un percorso di cartelle separate da “/”, chiamato percorso dei file
- I percorsi che iniziano con “/” sono chiamati percorsi assoluti
- iniziano la ricerca dalla radice del file system
- I percorsi che NON iniziano con “/” sono chiamati percorsi relativi
- iniziano la ricerca dalla directory corrente

Il comando pwd stampa la directory corrente

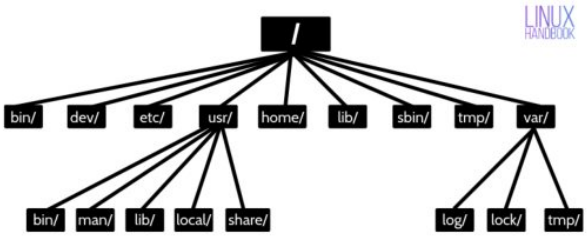
SLIDE ORIGINALE · diagrammi, tabelle e codice

I file

I file contengono altri file, che si diramano dalla radice “/” formando una gerarchia ad albero
- Si trovano in un percorso di cartelle separate da “/”, chiamato percorso dei file
- I percorsi che iniziano con “/” sono chiamati percorsi **assoluti**
  - iniziano la ricerca dalla radice del file system
- I percorsi che **NON** iniziano con “/” sono chiamati percorsi **relativi**
  - iniziano la ricerca dalla directory corrente

```
pwd
/Users/maurapintor
~
```

Il comando pwd stampa la directory corrente



saiferlab.ai

24

02 / SLIDE 025 Comandi utili

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 25 di 47

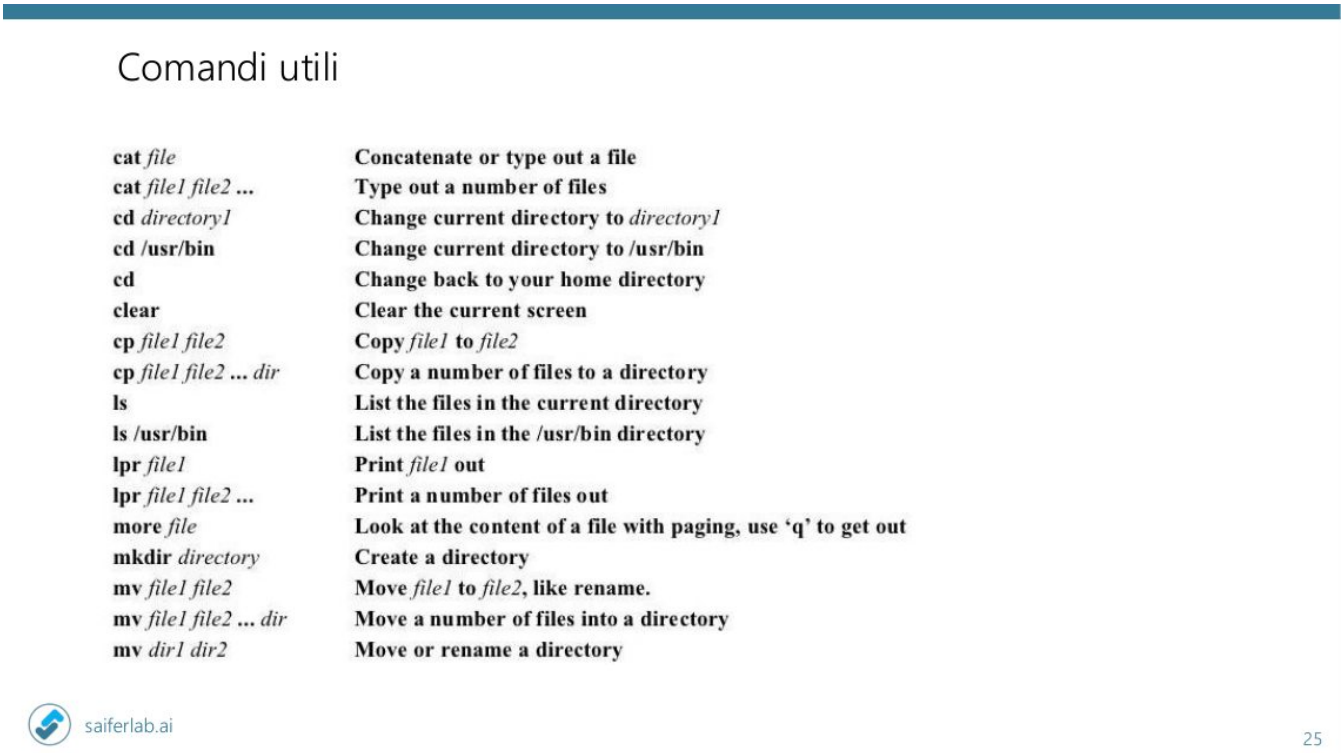
Spiegazione per studiare

La pagina riunisce i comandi utili della shell: è soprattutto una slide di consultazione. In fase di studio bisogna concentrarsi sullo scopo delle istruzioni e sugli effetti di lettura, navigazione, copia, spostamento o cancellazione, evitando l'esecuzione indiscriminata dei comandi distruttivi.

Definizioni, passaggi ed esempi presenti nella slide

Questa slide non contiene testo estraibile; la figura originale è disponibile nel riferimento visivo.

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 026 Il comando sudo rm -rf /: significato e rischi

Lezione: Strumenti, shell, Python e Git
Fonte: 02_strumenti_e_consigli.pdf, pagina 26 di 47

Spiegazione per studiare

La sequenza `sudo rm -rf /` combina l'esecuzione con privilegi elevati e una cancellazione ricorsiva e forzata a partire dalla radice. Il valore didattico è comprendere l'effetto della combinazione delle opzioni, non provare a eseguirla. È un esempio concreto dei rischi dell'automazione senza verifiche.

Definizioni, passaggi ed esempi presenti nella slide

Vediamo i pezzi che lo compongono:

• sudo serve a eseguire un comando come un altro utente (default come root, con permessi massimi)

• Il comando rm serve a rimuovere un file

• Il parametro r serve a rimuovere tutta la gerarchia di file che nasce dalla radice indicata come argomento

• Il parametro f serve a non chiedere conferma prima dell'eliminazione

• L'argomento / è il path radice

... Quindi questo comando cancellerà ogni file presente nel sistema!

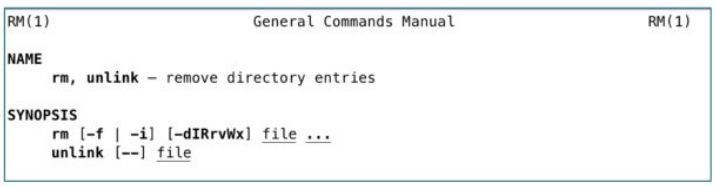
SLIDE ORIGINALE · diagrammi, tabelle e codice

Ma quindi, cosa fa il comando `sudo rm -rf /` ?

Vediamo i pezzi che lo compongono:

- **sudo** serve a eseguire un comando come un altro utente (default come root, con permessi massimi)
- Il comando **rm** serve a rimuovere un file
- Il parametro **r** serve a rimuovere tutta la gerarchia di file che nasce dalla radice indicata come argomento
- Il parametro **f** serve a non chiedere conferma prima dell'eliminazione
- L'argomento **/** è il path radice

... Quindi questo comando cancellerà ogni file presente nel sistema!



```
RM(1)                    General Commands Manual                    RM(1)

NAME
     rm, unlink – remove directory entries

SYNOPSIS
     rm [-f | -i] [-dIRrvWx] file ...
     unlink [--] file
```

saiferlab.ai

26

02 / SLIDE 027 Il linguaggio Python e il suo funzionamento

Lezione: Strumenti, shell, Python e Git
Fonte: 02_strumenti_e_consigli.pdf, pagina 27 di 47

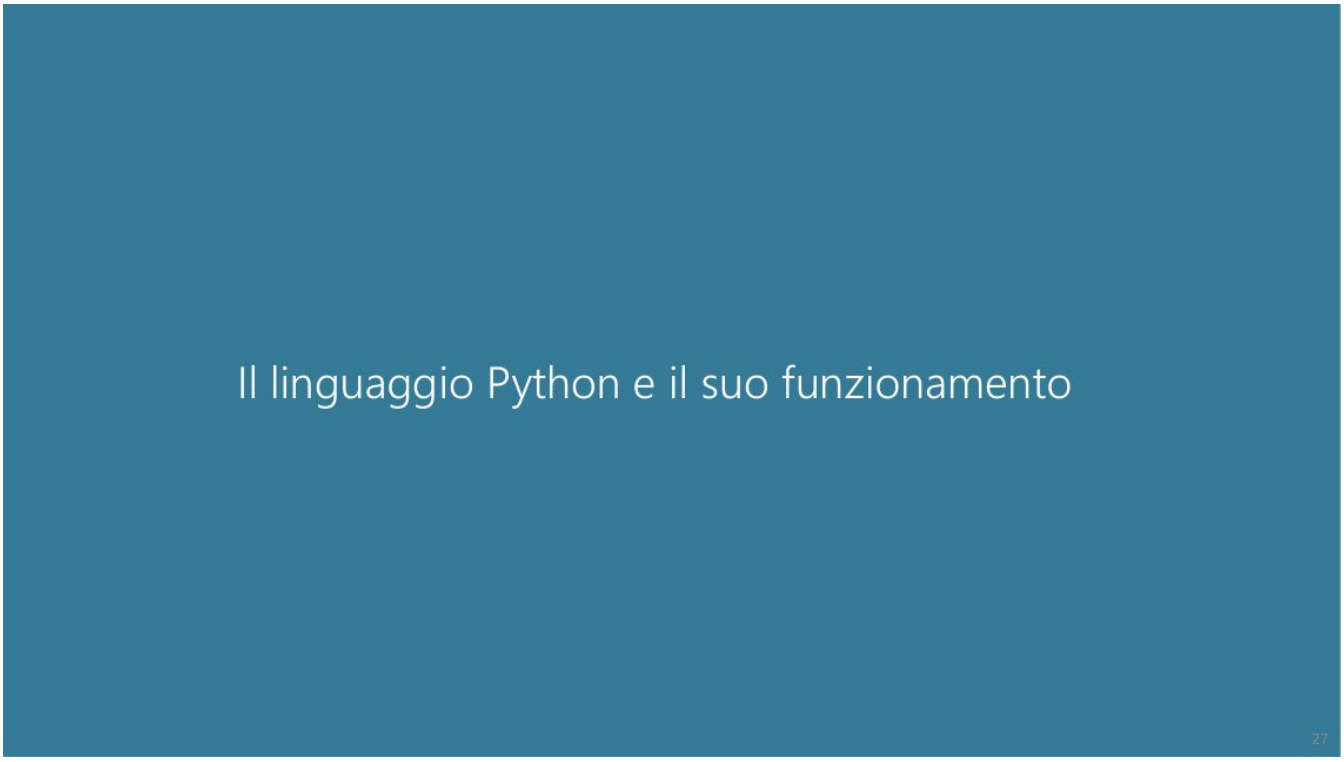
Spiegazione per studiare

Questa pagina introduce la sezione su Python e sui meccanismi di esecuzione del suo codice. Funziona come una pagina divisoria, quindi non aggiunge una nuova definizione da memorizzare.

Definizioni, passaggi ed esempi presenti nella slide

Il linguaggio Python e il suo funzionamento

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 028 Python

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 28 di 47

Spiegazione per studiare

Python è il linguaggio usato prevalentemente nel corso per il backend. È descritto come linguaggio interpretato e si presta a costruire servizi Web tramite librerie e framework. Occorre distinguere il codice applicativo dall'interprete e dall'ambiente che fornisce le dipendenze necessarie.

Definizioni, passaggi ed esempi presenti nella slide

Nel corso useremo Python per creare un web server (cosa sia un web server sarà chiaro più avanti)

• non è solitamente il linguaggio più usato per queste cose, ma ci aiuterà per capire le basi senza concentrarci sulle difficoltà di imparare linguaggi nuovi

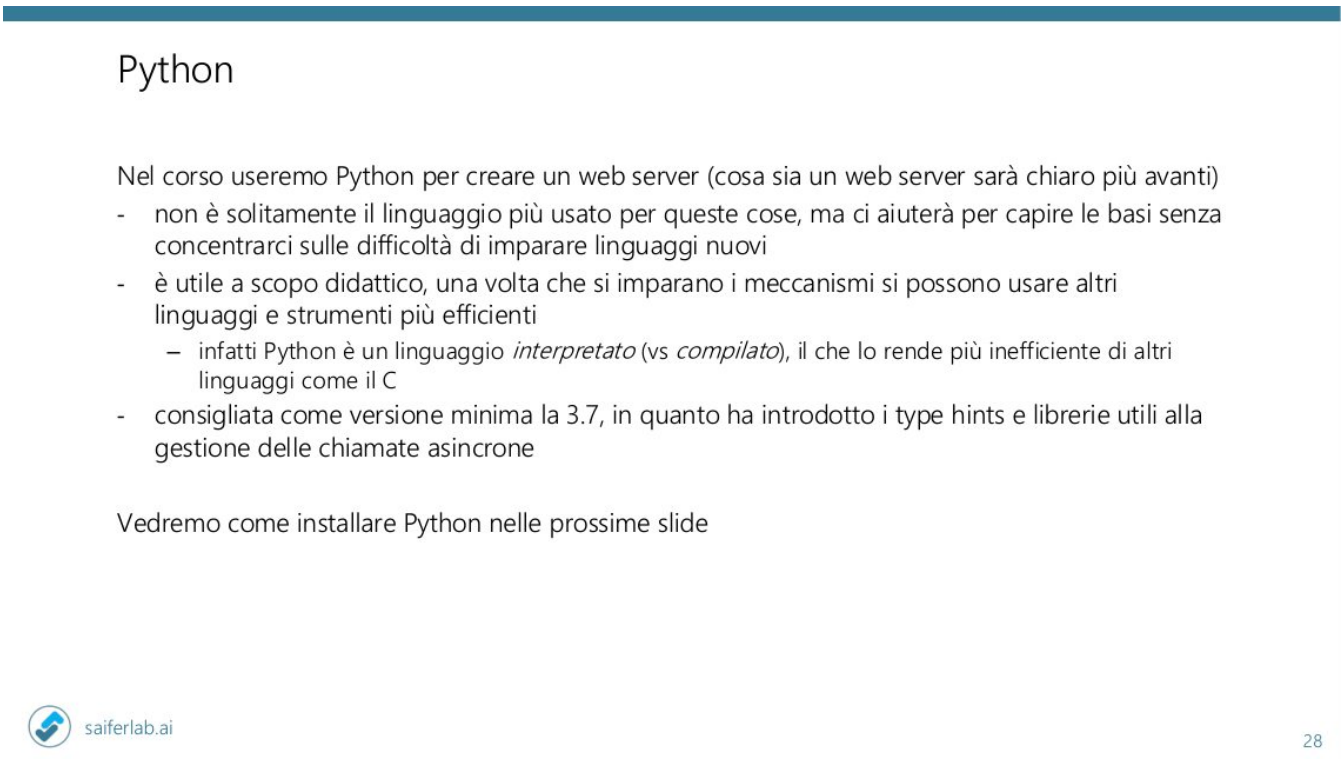
• è utile a scopo didattico, una volta che si imparano i meccanismi si possono usare altri linguaggi e strumenti più efficienti

• infatti Python è un linguaggio interpretato (vs compilato), il che lo rende più inefficiente di altri linguaggi come il C

• consigliata come versione minima la 3.7, in quanto ha introdotto i type hints e librerie utili alla gestione delle chiamate asincrone

Vedremo come installare Python nelle prossime slide

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 029 Gestione delle librerie Python

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 29 di 47

Spiegazione per studiare

Le librerie Python contengono funzionalità riutilizzabili e si installano con gestori di pacchetti come `pip`. Lo sviluppatore può quindi impiegare software già pronto per operazioni frequenti, mantenendo però il controllo delle versioni e delle dipendenze del progetto.

Definizioni, passaggi ed esempi presenti nella slide

```
Non scriveremo tutto il codice da zero, ma faremo uso di librerie software (
collezioni di funzioni e
classi scritte da altri e importabili nel nostro codice)
```

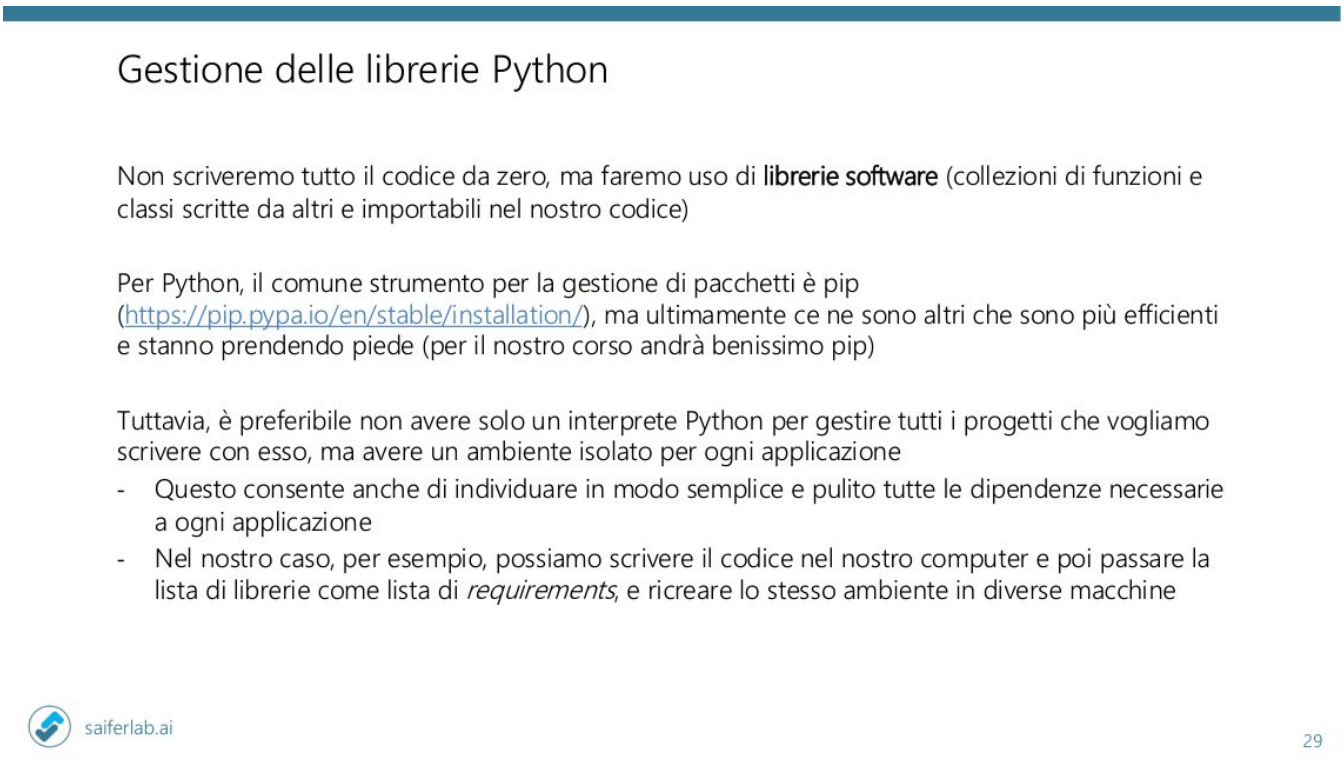
Per Python, il comune strumento per la gestione di pacchetti è pip (https://pip.pypa.io/en/stable/installation/), ma ultimamente ce ne sono altri che sono più efficienti e stanno prendendo piede (per il nostro corso andrà benissimo pip)

Tuttavia, è preferibile non avere solo un interprete Python per gestire tutti i progetti che vogliamo scrivere con esso, ma avere un ambiente isolato per ogni applicazione

• Questo consente anche di individuare in modo semplice e pulito tutte le dipendenze necessarie a ogni applicazione

• Nel nostro caso, per esempio, possiamo scrivere il codice nel nostro computer e poi passare la lista di librerie come lista di requirements, e ricreare lo stesso ambiente in diverse macchine

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 030 Ambienti Virtuali

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 30 di 47

Spiegazione per studiare

Gli ambienti virtuali isolano interprete e pacchetti utilizzati da progetti differenti. Questa separazione evita che l'installazione o l'aggiornamento di una dipendenza rompa un'altra applicazione. L'ambiente va creato e attivato prima di installare le librerie specifiche.

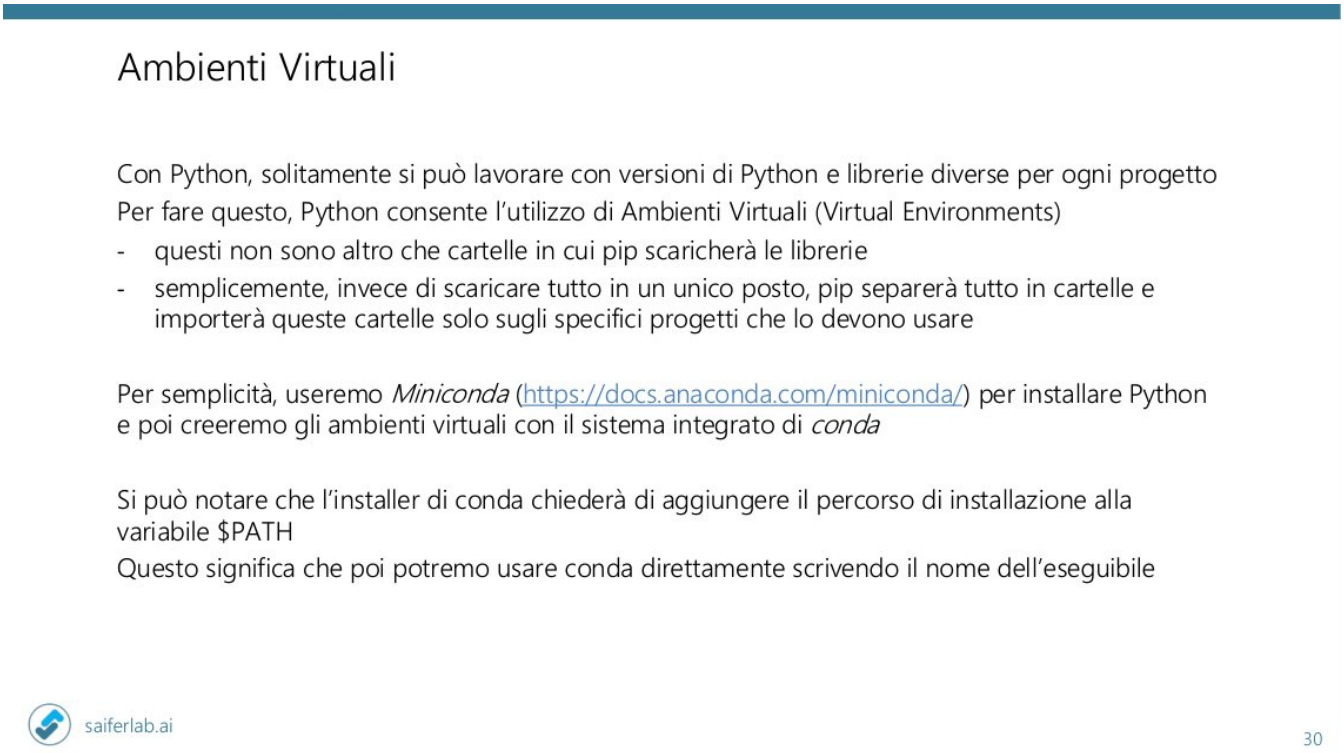
Definizioni, passaggi ed esempi presenti nella slide

```
Con Python, solitamente si può lavorare con versioni di Python e librerie
diverse per ogni progetto
Per fare questo, Python consente l’utilizzo di Ambienti Virtuali (Virtual
Environments)
-
questi non sono altro che cartelle in cui pip scaricherà le librerie
-
semplicemente, invece di scaricare tutto in un unico posto, pip separerà tutto
in cartelle e
importerà queste cartelle solo sugli specifici progetti che lo devono usare
```

Per semplicità, useremo Miniconda (https://docs.anaconda.com/miniconda/) per installare Python e poi creeremo gli ambienti virtuali con il sistema integrato di conda

Si può notare che l’installer di conda chiederà di aggiungere il percorso di installazione alla variabile $PATH Questo significa che poi potremo usare conda direttamente scrivendo il nome dell’eseguibile

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 031 Ambienti Virtuali

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 31 di 47

Spiegazione per studiare

La slide propone alcuni comandi di conda per creare e attivare un environment. Il nome dell'ambiente identifica il gruppo di dipendenze; l'eventuale versione di Python impostata ne determina l'interprete. Attivarlo modifica il contesto in cui vengono eseguiti i comandi Python successivi.

Definizioni, passaggi ed esempi presenti nella slide

Per creare l’environment

```
conda create -n myenv python=3.9
Per attivare l’environment
```

```
conda activate myenv
```

Ora, ogni volta che si installa un pacchetto con pip, questo lo scaricherà nella cartella dove risiede l’interprete virtuale “myenv”

SLIDE ORIGINALE · diagrammi, tabelle e codice

Ambienti Virtuali

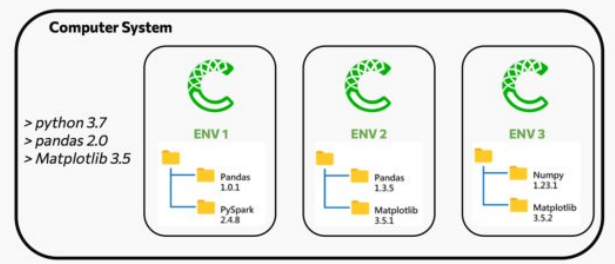
Per creare l’environment

```
conda create -n myenv python=3.9
```

Per attivare l’environment

```
conda activate myenv
```

Ora, ogni volta che si installa un pacchetto con pip, questo lo scaricherà nella cartella dove risiede l’interprete virtuale “myenv”



saiferlab.ai

31

02 / SLIDE 032 Ambienti Virtuali

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 32 di 47

Spiegazione per studiare

`conda env list` mostra gli ambienti disponibili e i relativi percorsi. La funzione pratica è verificare quale ambiente è presente e scegliere quello associato al progetto prima di avviare codice o installazioni.

Definizioni, passaggi ed esempi presenti nella slide

Si può chiedere a conda anche la lista di environment presenti

```
conda env list
Questo ci darà il percorso che contiene l’eseguibile python per ogni
environment e i relativi
pacchetti installati
```

```
conda env remove –n myenv
Quando non sono più necessari, gli environment possono anche essere rimossi
```

SLIDE ORIGINALE · diagrammi, tabelle e codice

Ambienti Virtuali

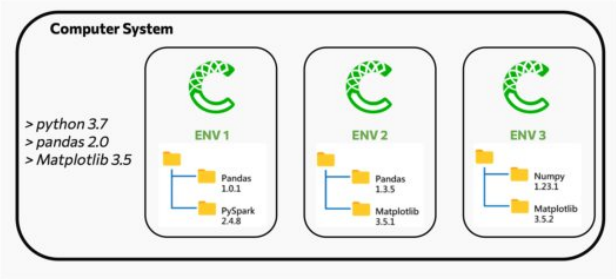
Si può chiedere a conda anche la lista di environment presenti

```
conda env list
```

Questo ci darà il percorso che contiene l’eseguibile python per ogni environment e i relativi pacchetti installati

```
conda env remove –n myenv
```

Quando non sono più necessari, gli environment possono anche essere rimossi



saiferlab.ai

32

02 / SLIDE 033 Ambienti Virtuali

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 33 di 47

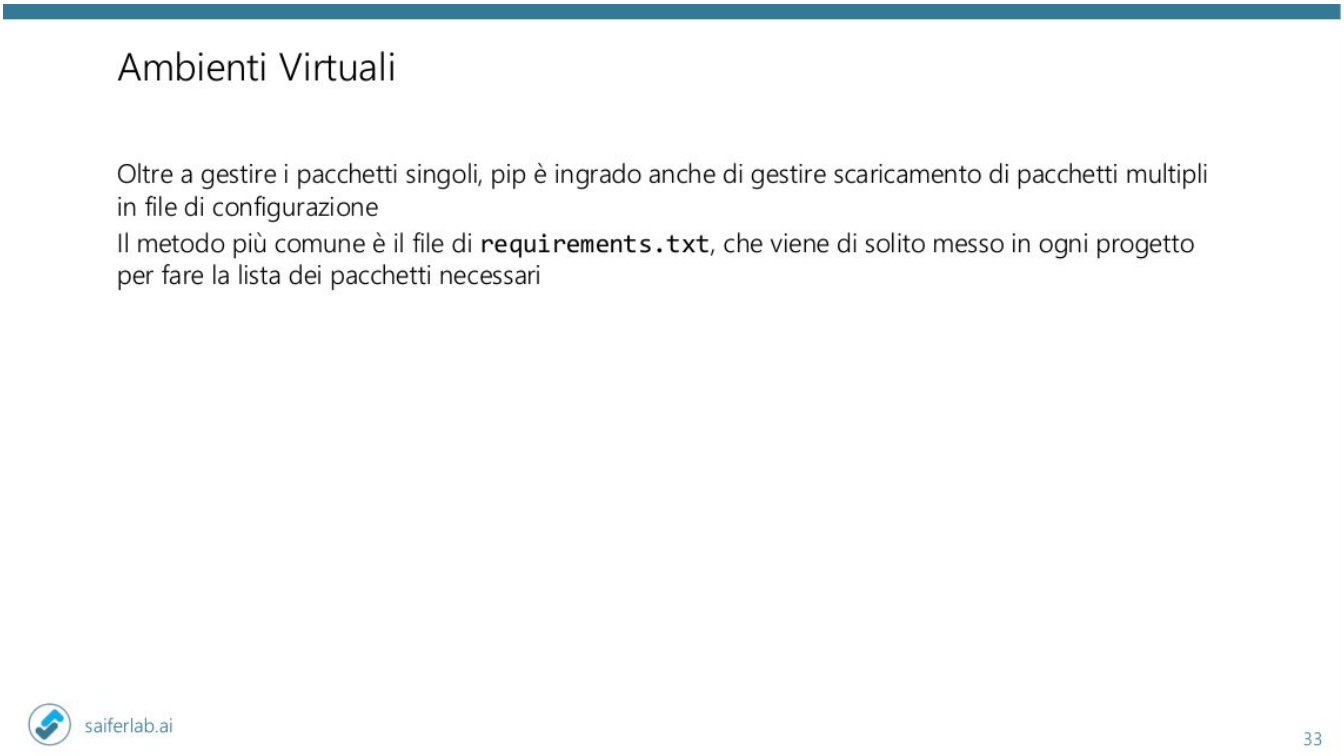
Spiegazione per studiare

Un file di requisiti permette di installare insieme i pacchetti necessari, rendendo più semplice riprodurre un ambiente su un altro computer. La gestione delle dipendenze è parte della riproducibilità: non basta condividere i sorgenti se mancano le librerie giuste.

Definizioni, passaggi ed esempi presenti nella slide

Oltre a gestire i pacchetti singoli, pip è ingrado anche di gestire scaricamento di pacchetti multipli in file di configurazione Il metodo più comune è il file di requirements.txt, che viene di solito messo in ogni progetto per fare la lista dei pacchetti necessari

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 034 Software utili e framework

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 34 di 47

Spiegazione per studiare

Il framework è un insieme coerente di componenti che fornisce funzionalità applicative già organizzate. Rispetto alle singole librerie, orienta anche la struttura del progetto e il modo in cui si integrano i componenti. Per il backend Web risulta utile nella gestione delle richieste e delle risposte.

Definizioni, passaggi ed esempi presenti nella slide

Un framework è una collezione di software che fornisce delle funzionalità generiche (nel nostro caso di sviluppo web) che possono essere cambiate e specializzate per risultare in software specifico per l'applicazione desiderata (per esempio, il nostro server web)

Per esempio, non dovremo scrivere tutto il codice per le richieste HTTP, questo sarà gestito dal framework, mentre noi ci dovremo occupare solo di scrivere le API per le richieste

In questo corso useremo una libreria Python popolare per la sua semplicità di utilizzo e flessibilità: FastAPI (https://fastapi.tiangolo.com)

SLIDE ORIGINALE · diagrammi, tabelle e codice

Software utili e framework

Un framework è una collezione di software che fornisce delle funzionalità generiche (nel nostro caso di *sviluppo web*) che possono essere cambiate e specializzate per risultare in software specifico per l'applicazione desiderata (per esempio, il nostro server web)

Per esempio, non dovremo scrivere tutto il codice per le richieste HTTP, questo sarà gestito dal framework, mentre noi ci dovremo occupare solo di scrivere le API per le richieste

In questo corso useremo una libreria Python popolare per la sua semplicità di utilizzo e flessibilità: FastAPI (https://fastapi.tiangolo.com)

02 / SLIDE 035 Gestione del codice e versionamento

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 35 di 47

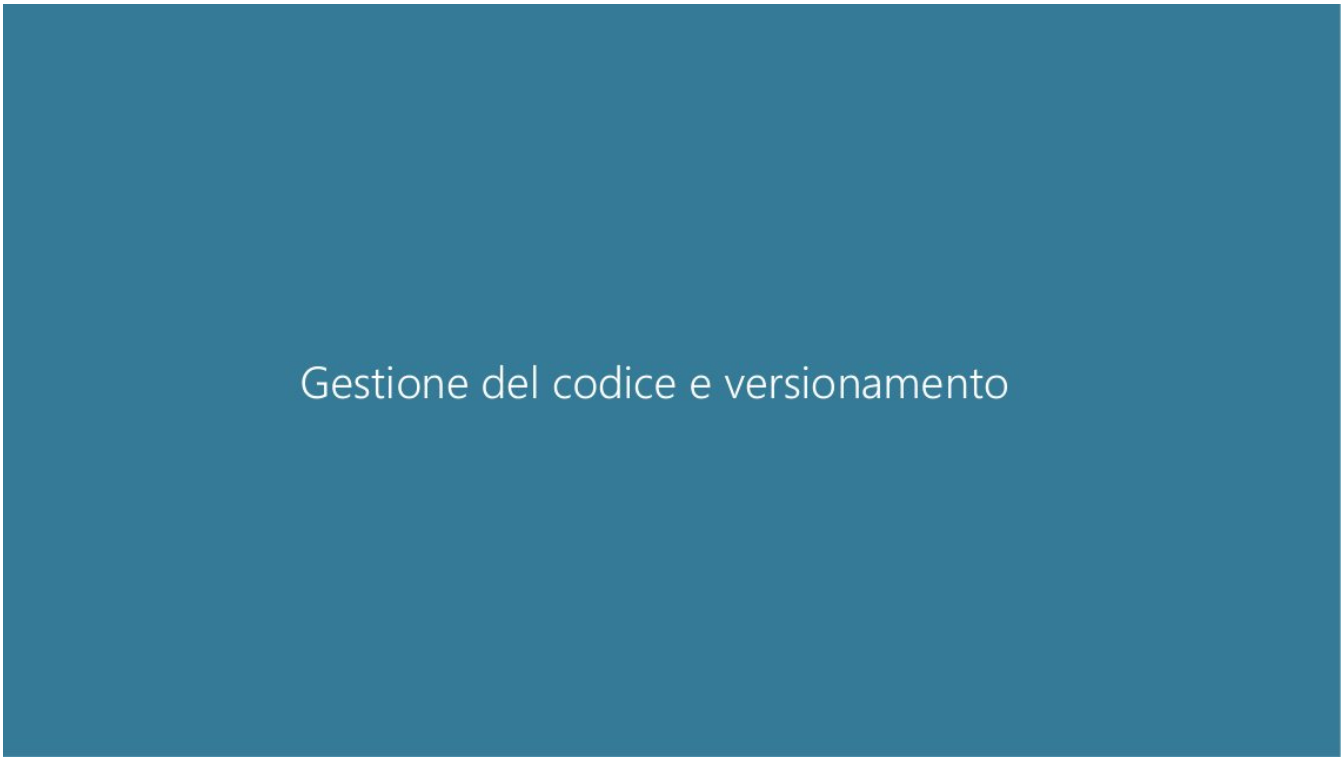
Spiegazione per studiare

La pagina segna il passaggio dagli strumenti di esecuzione del codice alle tecniche per gestirne le versioni e la collaborazione. È una slide introduttiva di sezione.

Definizioni, passaggi ed esempi presenti nella slide

Gestione del codice e versionamento

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 036 Importanza del versionamento

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 36 di 47

Spiegazione per studiare

Il versionamento serve a registrare modifiche, recuperare versioni precedenti e collaborare senza perdere il lavoro svolto. È particolarmente importante nei progetti Web, dove più funzionalità cambiano nel tempo e gli sviluppatori devono conoscere l'evoluzione del software.

Definizioni, passaggi ed esempi presenti nella slide

Il versionamento del codice è fondamentale per la gestione efficiente dei progetti software, consentendo di tenere traccia delle modifiche e facilitando il lavoro di team distribuiti

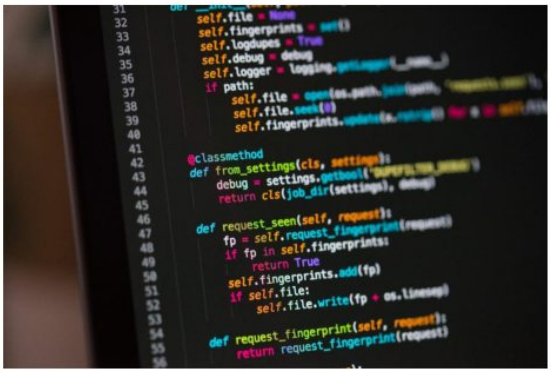
- Condivisione degli aggiornamenti
- Cronologia e tracciabilità delle modifiche (e possibilità di annullarle)
- Riproducibilità dei risultati (e dei bug)

SLIDE ORIGINALE · diagrammi, tabelle e codice

Importanza del versionamento

Il versionamento del codice è fondamentale per la gestione efficiente dei progetti software, consentendo di tenere traccia delle modifiche e facilitando il lavoro di team distribuiti

- Condivisione degli aggiornamenti
- Cronologia e tracciabilità delle modifiche (e possibilità di annullarle)
- Riproducibilità dei risultati (e dei bug)



saiferlab.ai

36

02 / SLIDE 037 Strumenti per il versionamento: Git

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 37 di 47

Spiegazione per studiare

Git è un sistema di controllo di versione che registra la storia dei file del progetto. La prima fase consiste nell'installarlo e configurarlo; dopo, si può inizializzare un repository e creare una sequenza tracciabile di modifiche.

Definizioni, passaggi ed esempi presenti nella slide

Installazione di Git e configurazione sono abbastanza semplici

- https://git-scm.com/downloads

In questo caso avere familiarità con la command line sarà d'aiuto Ci sono anche strumenti grafici (GUI) per la gestione dei repository

- GUI clients
- Ambienti di sviluppo integrato (IDE)

Vediamo i principali comandi con la command line

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 038 Concetti fondamentali

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 38 di 47

Spiegazione per studiare

Un repository contiene i file del progetto e lo storico delle revisioni. I commit rappresentano stati registrati; la staging area consente di scegliere quali modifiche includere nella revisione successiva. Questi concetti aiutano a distinguere file modificati, file preparati e file già salvati nella storia.

Definizioni, passaggi ed esempi presenti nella slide

Repository Un repository Git è un ambiente di lavoro che contiene tutti i file del progetto e la cronologia delle modifiche, sia locali che remoti

Commit Ogni commit rappresenta uno snapshot della tua applicazione, rendendo possibile il ripristino a una versione precedente in qualsiasi momento

- viene salvato solo “cosa è cambiato”, ovvero il “diff”

```
Branching e Merging
Git consente di creare branch per sviluppare funzionalità isolate,
mentre il merging integra queste modifiche nel branch
principale senza perdere la storia
```

SLIDE ORIGINALE · diagrammi, tabelle e codice

Concetti fondamentali

Repository

Un repository Git è un ambiente di lavoro che contiene tutti i file del progetto e la cronologia delle modifiche, sia locali che remoti

Commit

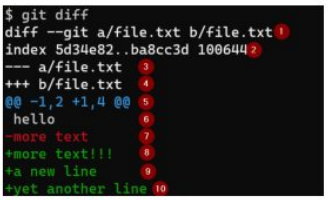
Ogni commit rappresenta uno snapshot della tua applicazione, rendendo possibile il ripristino a una versione precedente in qualsiasi momento

- viene salvato solo “cosa è cambiato”, ovvero il “*diff*”

Branching e Merging

Git consente di creare branch per sviluppare funzionalità isolate, mentre il merging integra queste modifiche nel branch principale senza perdere la storia





saiferlab.ai

38

02 / SLIDE 039 Comandi iniziali di Git

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 39 di 47

Spiegazione per studiare

I primi comandi Git permettono di inizializzare un repository, selezionare modifiche e salvarle in commit. L'operazione di commit dovrebbe corrispondere a un cambiamento comprensibile, così che sia più semplice ricostruire il motivo di ogni intervento.

Definizioni, passaggi ed esempi presenti nella slide

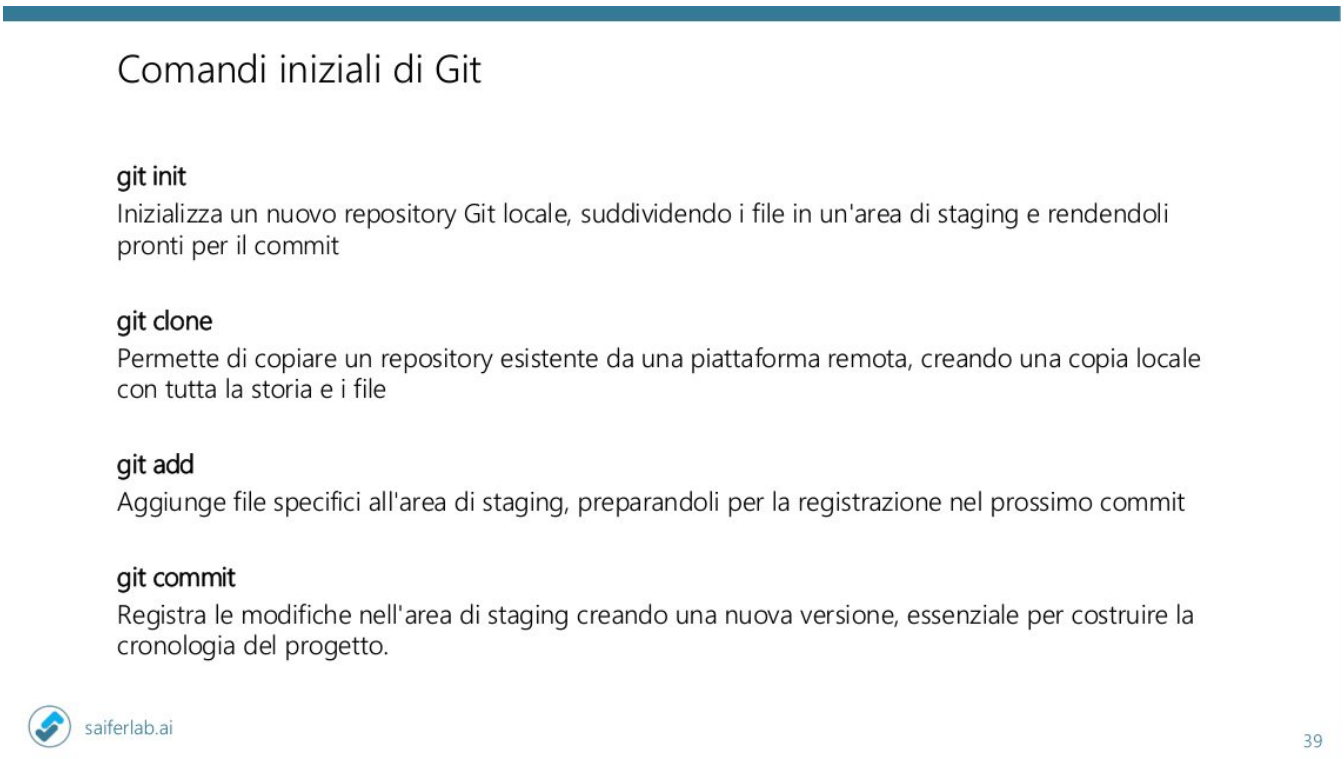
```
git init
Inizializza un nuovo repository Git locale, suddividendo i file in un'area di
staging e rendendoli
pronti per il commit

git clone
Permette di copiare un repository esistente da una piattaforma remota, creando
una copia locale
con tutta la storia e i file

git add
Aggiunge file specifici all'area di staging, preparandoli per la registrazione
nel prossimo commit

git commit
Registra le modifiche nell'area di staging creando una nuova versione,
essenziale per costruire la
cronologia del progetto.
```

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 040 Gestione dei branch

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 40 di 47

Spiegazione per studiare

Un branch è una linea di sviluppo separata: si possono implementare funzionalità o sperimentare modifiche senza agire direttamente sul ramo principale. Successivamente il lavoro può essere integrato con le opportune operazioni di unione.

Definizioni, passaggi ed esempi presenti nella slide

In Git abbiamo anche la possibilità di creare dei ”rami” (branch) indipendenti di lavoro

• Un branch principale (main) dove risiede la cronologia principale

• Si possono creare anche branch generati da altri branch

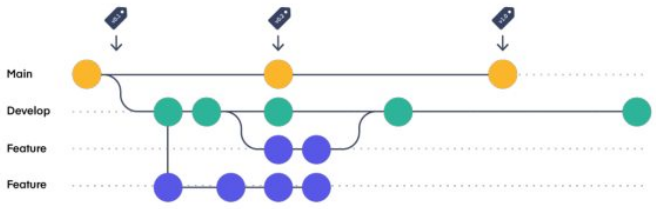
• Passare da un branch all’altro aggiorna la cartella contenente il repository allo stato attuale di ogni branch (con le modifiche locali)

SLIDE ORIGINALE · diagrammi, tabelle e codice

Gestione dei branch

In Git abbiamo anche la possibilità di creare dei ”rami” (branch) indipendenti di lavoro

- Un branch principale (*main*) dove risiede la cronologia principale
  - Si possono creare anche branch generati da altri branch
  - Passare da un branch all’altro aggiorna la cartella contenente il repository allo stato attuale di ogni branch (con le modifiche locali)



saiferlab.ai

40

02 / SLIDE 041 Gestione dei branch

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 41 di 47

Spiegazione per studiare

I branch permettono a più sviluppatori di procedere contemporaneamente. La slide ne collega l'uso alle nuove funzionalità e alle prove sperimentali. La loro gestione richiede di capire come creare, cambiare e integrare i diversi rami di lavoro.

Definizioni, passaggi ed esempi presenti nella slide

I branch sono spesso usati per collaborazioni con più sviluppatori

• Usati anche per provare le funzionalità che si aggiungono prima di fare modifiche al ramo principale

• L'operazione di unione al branch principale si chiama merge

• Cosa succede se più persone/commit modificano lo stesso file?

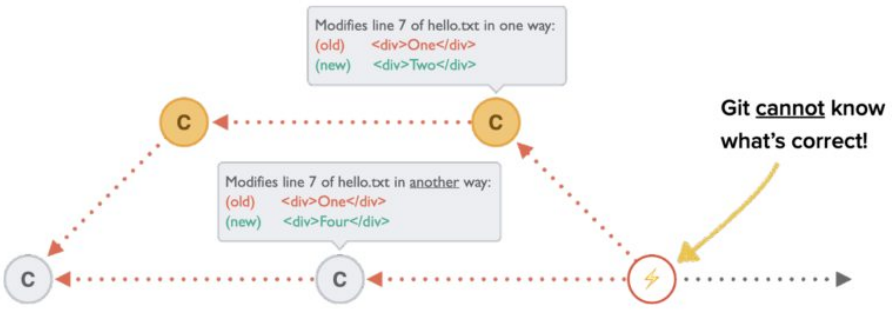
• I conflitti devono essere risolti manualmente, richiedendo attenzione ai dettagli e conoscenza del codice

SLIDE ORIGINALE · diagrammi, tabelle e codice

Gestione dei branch

I branch sono spesso usati per collaborazioni con più sviluppatori

- Usati anche per provare le funzionalità che si aggiungono *prima* di fare modifiche al ramo principale
- L'operazione di unione al branch principale si chiama **merge**
  - Cosa succede se più persone/commit modificano lo stesso file?
  - I **conflitti** devono essere risolti manualmente, richiedendo attenzione ai dettagli e conoscenza del codice



saiferlab.ai

41

02 / SLIDE 042 Condivisione del codice e repository online: Github

Lezione: Strumenti, shell, Python e Git
Fonte: 02_strumenti_e_consigli.pdf, pagina 42 di 47

Spiegazione per studiare

GitHub aggiunge a Git un servizio remoto per pubblicare repository e collaborare. Consente di condividere il lavoro, coordinare le modifiche e mantenere una copia del progetto raggiungibile dagli altri membri del gruppo.

Definizioni, passaggi ed esempi presenti nella slide

GitHub (https://github.com) è una popolare piattaforma di collaborazione basata su Git

Fornisce hosting per repository e strumenti di collaborazione per progetti di sviluppo

Mentre Git è un sistema di controllo versione, GitHub fornisce la capacità di archiviare repository remoti e gestire collaborazioni su larga scala

GitHub offre strumenti per la revisione del codice, gestione delle issue e integrazioni con CI/CD, semplificando il flusso di lavoro degli sviluppatori (prossima slide)

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 043 Le issue di GitHub

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 43 di 47

Spiegazione per studiare

Le issue di GitHub raccolgono segnalazioni di problemi, richieste di miglioramento e discussioni relative al progetto. Servono a rendere esplicite le attività da svolgere e a collegarle, quando necessario, alle modifiche del codice.

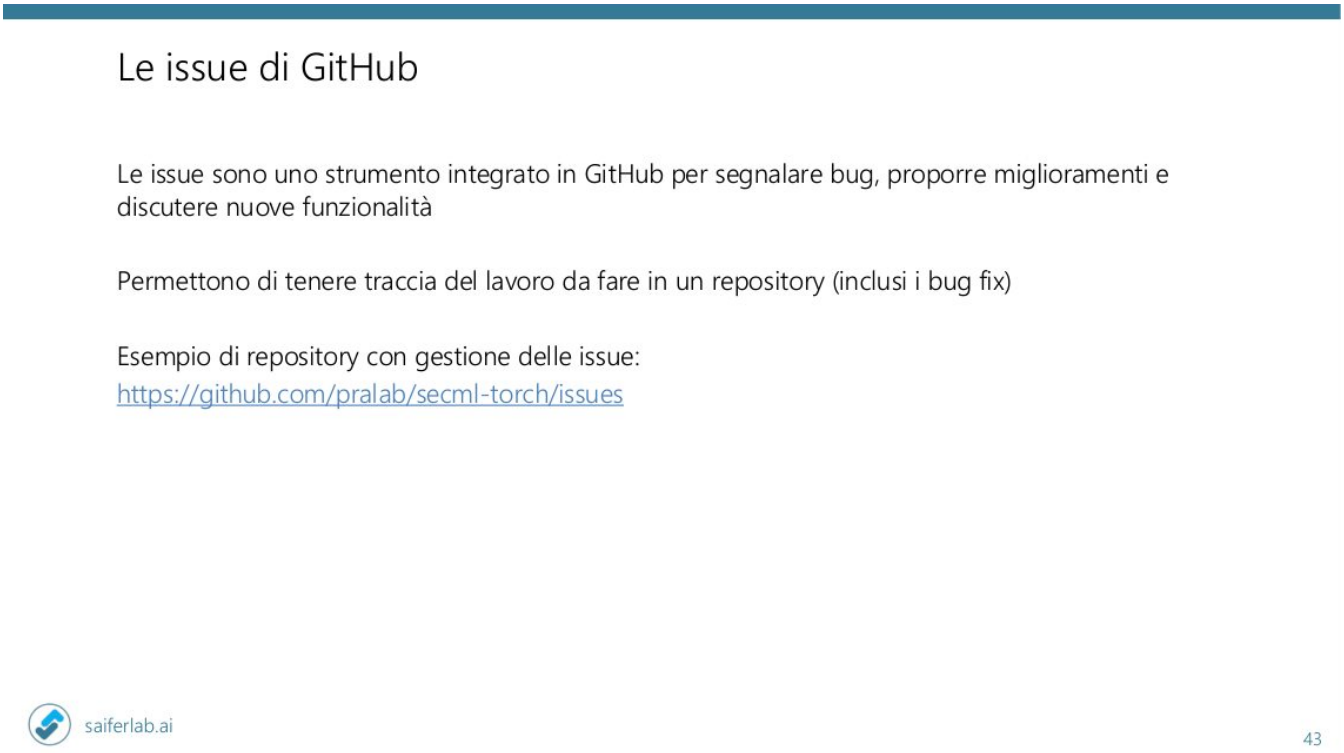
Definizioni, passaggi ed esempi presenti nella slide

Le issue sono uno strumento integrato in GitHub per segnalare bug, proporre miglioramenti e discutere nuove funzionalità

Permettono di tenere traccia del lavoro da fare in un repository (inclusi i bug fix)

Esempio di repository con gestione delle issue: https://github.com/pralab/secml-torch/issues

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 044 La Continuous Integration (CI) e il Continuous Deployment (CD)

Lezione: Strumenti, shell, Python e Git
Fonte: 02_strumenti_e_consigli.pdf, pagina 44 di 47

Spiegazione per studiare

La Continuous Integration automatizza controlli come build e test a seguito di modifiche al repository. La Continuous Deployment/Delivery riguarda invece il percorso con cui il software viene preparato o pubblicato. Questa automazione migliora ripetibilità e tracciabilità del rilascio.

Definizioni, passaggi ed esempi presenti nella slide

Continuous Integration (CI): Automatizza la verifica del codice a ogni modifica, eseguendo test e build in un ambiente controllato

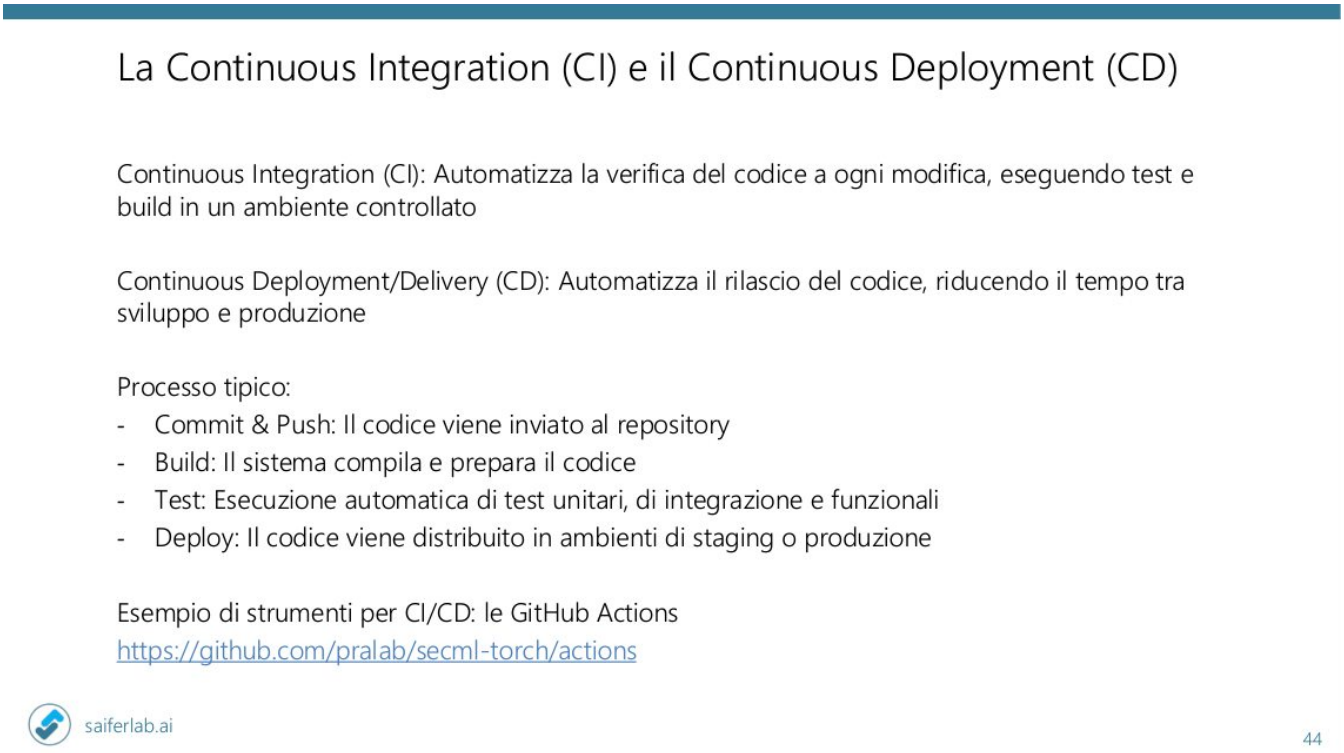
Continuous Deployment/Delivery (CD): Automatizza il rilascio del codice, riducendo il tempo tra sviluppo e produzione

Processo tipico:

- Commit & Push: Il codice viene inviato al repository
- Build: Il sistema compila e prepara il codice
- Test: Esecuzione automatica di test unitari, di integrazione e funzionali
- Deploy: Il codice viene distribuito in ambienti di staging o produzione

Esempio di strumenti per CI/CD: le GitHub Actions https://github.com/pralab/secml-torch/actions

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 045 Collegare un repository locale a GitHub

Lezione: Strumenti, shell, Python e Git
Fonte: 02_strumenti_e_consigli.pdf, pagina 45 di 47

Spiegazione per studiare

Collegare un repository locale a un remote permette di inviare e ricevere commit rispetto a GitHub. Le operazioni `push` e `pull` hanno ruoli diversi: la prima pubblica le revisioni locali, mentre la seconda incorpora quelle disponibili sul remoto.

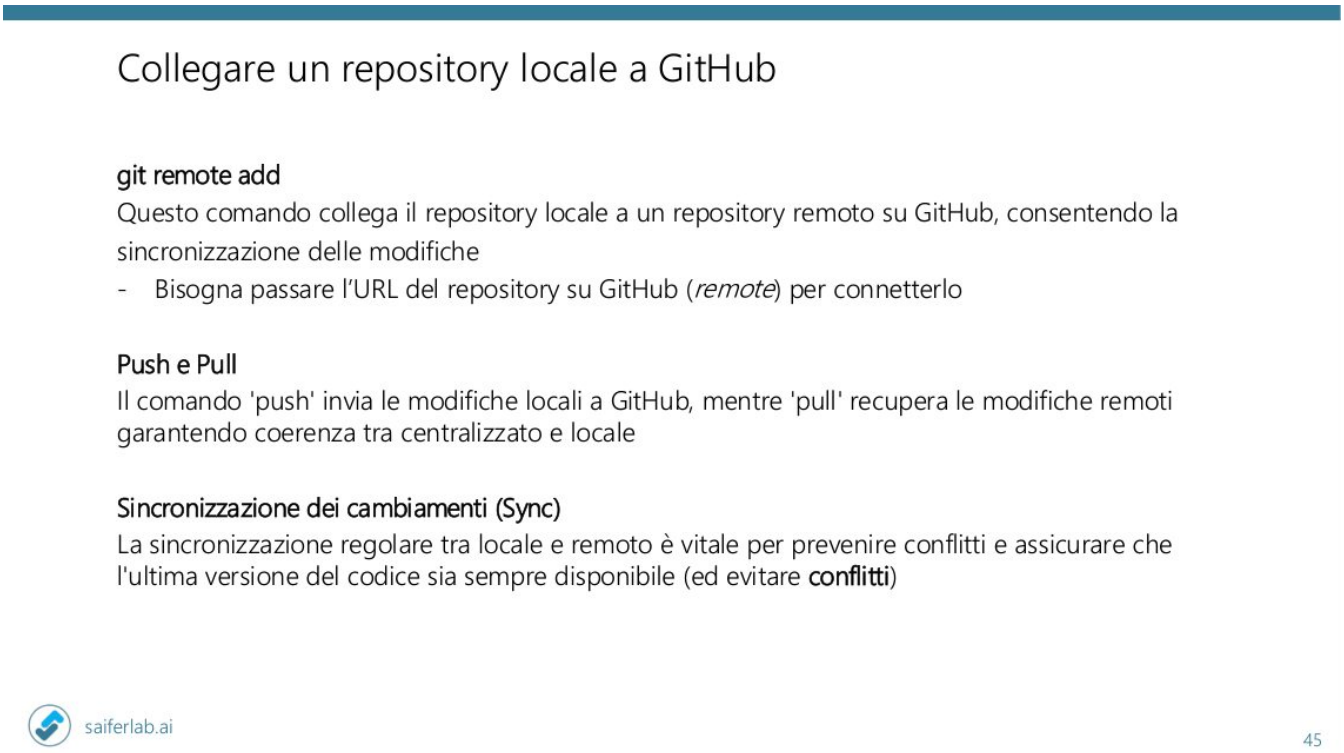
Definizioni, passaggi ed esempi presenti nella slide

```
git remote add
Questo comando collega il repository locale a un repository remoto su GitHub,
consentendo la
sincronizzazione delle modifiche
-
Bisogna passare l’URL del repository su GitHub (remote) per connetterlo
```

Push e Pull Il comando 'push' invia le modifiche locali a GitHub, mentre 'pull' recupera le modifiche remoti garantendo coerenza tra centralizzato e locale

Sincronizzazione dei cambiamenti (Sync) La sincronizzazione regolare tra locale e remoto è vitale per prevenire conflitti e assicurare che l'ultima versione del codice sia sempre disponibile (ed evitare conflitti)

SLIDE ORIGINALE · diagrammi, tabelle e codice



02 / SLIDE 046 Collaborazione su GitHub

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 46 di 47

Spiegazione per studiare

Il forking produce una copia del repository per lavorare su modifiche indipendenti; le pull request consentono di proporre che vengano integrate nel progetto di riferimento. È un modello frequente nella collaborazione su software condiviso.

Definizioni, passaggi ed esempi presenti nella slide

Forking Il forking permette di creare una copia del repository originale, consentendo modifiche personali senza influenzare l'originale (su cui spesso non si hanno i permessi di scrittura)

• Usato per sviluppare una feature indipendente in un proprio account GitHub e poi richiedere

l’aggiunta in un repository principale

Pull Requests Le pull request consentono agli sviluppatori di proporre modifiche al progetto originale, facilitando la revisione da parte degli altri collaboratori https://github.com/pralab/secml-torch/pulls

SLIDE ORIGINALE · diagrammi, tabelle e codice

Collaborazione su GitHub

Forking

Il forking permette di creare una copia del repository originale, consentendo modifiche personali senza influenzare l'originale (su cui spesso non si hanno i permessi di scrittura)

- Usato per sviluppare una feature indipendente in un proprio account GitHub e poi richiedere l’aggiunta in un repository principale

Pull Requests

Le pull request consentono agli sviluppatori di proporre modifiche al progetto originale, facilitando la revisione da parte degli altri collaboratori

https://github.com/pralab/secml-torch/pulls

saiferlab.ai

46

02 / SLIDE 047 Best practices per Git e GitHub

Lezione: Strumenti, shell, Python e Git

Fonte: 02_strumenti_e_consigli.pdf, pagina 47 di 47

Spiegazione per studiare

Le buone pratiche suggerite includono commit frequenti e significativi, messaggi chiari e uso disciplinato dei branch. Queste abitudini non cambiano il funzionamento di Git, ma rendono il progetto più facile da capire, verificare e mantenere.

Definizioni, passaggi ed esempi presenti nella slide

Commit frequenti Effettuare commit frequenti per ogni modifica incrementale contribuisce a mantenere una cronologia chiara e gestibile

Messaggi di commit chiari Scrivere messaggi di commit informativi e descrittivi è fondamentale per facilitare la comprensione delle modifiche apportate

Documentazione del codice Fornire documentazione chiara e dettagliata migliora notevolmente la possibilità di collaborare efficacemente e riduce il tempo necessario per comprendere il codice

SLIDE ORIGINALE · diagrammi, tabelle e codice

